Creating Personalized Systems that People Can Scrutinize and Control: Drivers, Principles and Experience

JUDY KAY and BOB KUMMERFELD, University of Sydney

Widespread personalized computing systems play an already important and fast-growing role in diverse contexts, such as location-based services, recommenders, commercial Web-based services, and teaching systems. The personalization in these systems is driven by information about the user, a *user model*. Moreover, as computers become both ubiquitous and pervasive, personalization operates across the many devices and information stores that constitute the user's personal digital ecosystem. This enables personalization, and the user models driving it, to play an increasing role in people's everyday lives. This makes it critical to establish ways to address key problems of personalization related to *privacy*, *invisibility* of personalization, *errors in user models*, *wasted user models*, and the broad issue of enabling people to *control* their user models and associated personalization. We offer *scrutable user models* as a foundation for tackling these problems.

This article argues the importance of scrutable user modeling and personalization, illustrating key elements in case studies from our work. We then identify the broad roles for scrutable user models. The article describes how to tackle the technical and interface challenges of designing and building scrutable user modeling systems, presenting design principles and showing how they were established over our twenty years of work on the Personis software framework. Our contributions are the set of principles for scrutable personalization linked to our experience from creating and evaluating frameworks and associated applications built upon them. These constitute a general approach to tackling problems of personalization by enabling users to scrutinize their user models as a basis for understanding and controlling personalization.

Categories and Subject Descriptors: H.2.1 [**Information Systems**]: User/Machine Systems

General Terms: Design, Algorithms, Performance

Additional Key Words and Phrases: Personalization, user model, user modeling, scrutability, understandability, user control, augmented cognition, open learner modeling, reasoning under uncertainty, inconsistency

ACM Reference Format:
Kay, J. and Kummerfeld, B. 2012. Creating personalized systems that people can scrutinize and control: Drivers, principles and experience. ACM Trans. Interact. Intell. Syst. 2, 4, Article 24 (December 2012), 42 pages.
DOI = 10.1145/2395123.2395129 http://doi.acm.org/10.1145/2395123.2395129

1. INTRODUCTION

Personalized interaction has become possible because it is increasingly easy to collect large amounts of information about people and to exploit it to improve interaction with personalization. Even ten years ago, there were many personalized services available

The reviewing of this article was managed by associate editor Riichiro Mizoguchi.
This work is supported by the Australian Research Council Discovery Grant DP0877665, Pervasive Lifelong User Modelling for User Controlled Personalisation and Augmented Cognition. Work reported in this article was supported by the Smart Internet Technology Cooperative Research Center, Smart Services Cooperative Research Center, and the Australian Research Council Discovery Awards.
Authors' addresses: J. Kay (corresponding author), B. Kummerfeld, School of Information Technologies, University of Sydney, J12, Australia; email: judy.kay@gmail.com.

DOI 10.1145/2395123.2395129 http://doi.acm.org/10.1145/2395123.2395129

on the Web [Kobsa et al. 2001] and the trend has increased steadily [Pariser 2011]. Personalization is likely to increase further, both in its extent on the Web and in other applications and services that operate on many devices. As computer hardware has become less expensive and networking and Web access have become widespread, many people already have a whole collection of devices and information stores that constitute their *personal digital ecosystem*. Its increasing numbers of diverse and powerful devices include desktops, as well as mobile devices, such as phones, tablets, and laptops, and emerging embedded devices, like tabletops, wall displays, and digital appliances. This ecosystem can accumulate huge numbers of diverse forms of personal digital artifacts, such as documents, images, video, email, calendars, contacts, blog posts, tweets, and posts at social network sites. It can also accumulate logs of user activity, such as Web page visits, Web search queries, accesses to digital artifacts as well as data collected by appliances and sensors that make it easy for the user to capture information such as weight or activity[1].

From the combination of personal data and logs of user activity, it is possible to create *user models*. Early definitions of this term described it as a set of beliefs about the user [Kobsa and Wahlster 1989] and emphasized that it is a distinct part of a personalized application or system, rather than embedded within the logic of the personalization code. Since then, the large body of user modeling work has included diverse forms of user models. These range from simple collections of flags about the user to sophisticated models that aim to simulate the user's reasoning. User models represent aspects such as: knowledge, interests, goals and tasks, background, individual traits and context [Brusilovsky and Millán 2007]. For example, one could build a model of the user's friends by analysis of one or more of: the user's page at social network sites, email, diary, and phone contacts. That model may be a simple list of names, or a more complex representation that aims to model the strength and nature of each relationship. Consider, as another example, a system that teaches the user mathematics. Each time the user loads a new page of tutorial material, the system could infer a greater likelihood the user knows that material. As the learner does assessment tasks, each of these could alter the part of the model representing aspects tested. Such user models drive the personalization in many classes of systems including personalized ecommerce, Web and desktop search engines, recommenders, personal information management tool advisors, help systems, and teaching systems [Brusilovsky et al. 2007] as well as emerging pervasive computing environments. These drive personalization that has the potential to improve the efficiency and effectiveness of interaction.

This article is concerned with *long-term user models*. These collect data about a person over a long period of time. Already, there have been many examples of deployed systems that build and use such models over several years. Some of the well-known examples include commercial Web-based services such as Web search engines, customized advertisements on many Web sites, and recommenders for diverse products. Notable among recommender systems is the very early, and now well-established, Amazon which began with books and diversified to many other products. Others include Netflix for movies, Facebook, and Google, which stores details such as the user's search history. Desktop tools, such as "slife"[2] track the user's activity on her own computer. This can help the user monitor her time management practices. Mobile applications, such as Google Latitude, maintain a location history for a user. In addition, there are many emerging devices and pervasive computing applications that create such models. For

[1]For example, the Withings scales, www.withings.com/, activity monitors like DirectLife http://www.directlife.philips.com/, Garmin FR60 http://www.garmin.com/, Nike sensor with ipod http://store.apple.com/au/product/MA368ZM/D.

[2]http://www.slifeweb.com/.

example, one can wear a small activity-sensing device[3]. This can capture fine-grained data such as steps walked and degree of activity-inactivity. Such devices are increasingly designed to make it very easy for this data to be transferred to a repository. The user can then review her level of activity over the long term.

These models can, and already do, drive personalization in many commercial Web-based systems and there is emerging work in pervasive computing applications that will personalize many other aspects. Broadly, there is a trend towards the capture of large amounts of user model information, from the diverse set of devices that are in the user's personal digital ecosystem. This information is held both within the many storage elements of that ecosystem and within the stores of service providers. Since it can be used directly for personalization, this information can be seen as a form of user model, albeit often simple but large. Equally, it can serve as evidence for building user models. Indeed, companies such as Acxiom, bluekai, Zoominfo, Infogroup, and pipl[4] are aggregating data about people, from a range of sources for use in various forms of personalization and filtering. Widespread collection of personal data and personalization comes with some significant problems.

—*Privacy*. User models that drive personalization are personal information. Accordingly, they are subject to the complex mix of national and international legislation concerning proper management of personal information. This has a common requirement that people should have *access to and control over* their own data [Kobsa 2002, 2007; Iachello and Hong 2007; Wang and Kobsa 2007, 2009]. In current widespread personalized systems, the norm is that people can neither access nor control their user models. The legislation reflects the broad agreement that this is a problem. It is particularly so for the significant minority of people who have high levels of concern about privacy [Chellappa and Sin 2005; Iachello and Hong 2007]. There is special concern about personalization associated with pervasive and ubiquitous computing [Langheinrich 2002; Price et al. 2005] and mobile phones [Shilton 2009]. It is notable that people appear to have become increasingly concerned about their privacy being compromised in personalized systems [Westin 2003; Toch et al. 2012].

—*Invisibility*. For the most part, current widespread personalization operates invisibly. It is at best difficult, and often impossible, for people to determine whether an interface is personalized (or what aspects are personalized). This means people cannot control personalization applications and services. So, for example, one may receive a highly filtered view of the world via personalized news services without realizing that this is so. Similarly, when two users visit the same Web site, or use the same application, they may not realize that they get different results because of personalization. Considerable public discussion has echoed concern about this risk of personalization and its potential to create "filter bubbles" [Pariser 2011].

—*Errors in user models*. Most widespread user modeling relies on noisy and uncertain information about the user. This is partly because the raw evidence often comes from "observations" of the user, for example, the links she appears to click on a Web site. It also comes from the complex inference processes driving many systems. These sources of errors mean that user models commonly contain errors. Moreover, people typically have no systematic way to fix these. People may resort to "hunting" behaviors [Browne et al. 1990], as both the system and user attempt to model each other. For example, a *Wall Street Journal* article [Zaslow 2002] reported several cases where a TiVo created incorrect models, such as inferring the user was gay. It

[3]such as http://www.fitbit.com/.

[4]http://www.acxiom.com/, http://pipl.com/, http://www.bluekai.com/aboutus.php, http://www.zoominfo.com/, http://www.infogroup.com/.

then gave poor recommendations. The article reported how people tried to overcome this by trying to find actions that would influence the system to alter its model. There are many sources of errors. For example, if someone else uses your account or computer, their activity may contribute to your user model, making it inaccurate.

—*Wasted user models*. Many devices capture large amounts of user modeling information about people, storing it in various computers, notably ones not owned or controlled by the user. Much of it lies dormant even though it could be useful to the user. For example, if a person buys a pedometer that wirelessly sends his data to the manufacturer's site, the user can only access it there. He cannot readily combine it with his other personal information to gain a fuller view of his fitness efforts and progress.

—*Control*. In current systems, people are simply not in control of either the personal information that constitutes their user models or the personalization based on them. This is a core problem that underlies several of those given before. In addition, there is evidence that people willingly provide more personal information for personalization if they feel that they are in control [Kobsa 2007; Toch et al. 2012].

In the next section, we introduce *scrutable user modeling* as a foundation for tackling the preceding problems. In Section 3, we present a series of case studies from our work to illustrate our vision for scrutable personalization. It serves as an introduction to Section 4 which describes the roles for scrutable user models. Then we present a set of broad principles for creation of scrutable user models and personalization in Section 5, followed by an overview of our Personis implementations of these (Section 6). We conclude with reflections on the current state of scrutable personalization and broad issues for making scrutability the norm for personalized systems.

2. INTRODUCTION TO SCRUTABILITY

Currently, most personalization is inscrutable, meaning that the user has no way to discover the details of her user model and the associated personalization. *Scrutable*[5] user models are designed and implemented so that the user can study, or *scrutinize*, the way she works, to determine what information the user model holds, the processes used to capture it, and the ways that it is used. It is similar to the terms *open* [Bull et al. 1995; Self 1999; Dimitrova 2003; Mitrovic and Martin 2006; Zapata-Rivera et al. 2007], *inspectable* [Zapata-Rivera and Greer 2004], *transparent* [Cramer et al. 2008; Hook 2000; Hook et al. 1996], and *intelligible* [Lim et al. 2009; Lim and Dey 2009]. We chose the term scrutable because it indicates the real effort that a user must make as she studies information about the user model and the personalization processes, in order to understand them. It is closest in meaning to intelligible and may be considered to be a particularly comprehensive form of open, inspectable, and transparent model. We now illustrate the notion of scrutable user modeling with two scenarios.

Scenario 1

Jim uses an editor inefficiently as he is unaware of its *undo* command. Its coach tells him about *undo*. Jim tries it and continues to use it.

Scenario 2

Mynews, Ann's personalized newspaper, presents headlines. She is surprised that today's top one is *Greens Party Politician Misuses Travel Allowance*.

Scenario 1 worked well, with the coach giving helpful advice based on its model of Jim's knowledge (and, in particular, lack of knowledge of *undo*). It used this to select a good teaching goal, *undo*, and presented it well. This scenario is in a long-term learning context, one that is relevant to the many productivity tools that play such an important role in the workplace. It is reminiscent of Microsoft's Clippy [Horvitz et al.

[5]"Capable of being understood through study and observation" http://www.yourdictionary.com/scrutable.

1998], arguably the first deployment of individual user modeling to large populations of Excel and Word users. It differs from our vision of scrutable user modeling. Clippy was not scrutable. The user could not see, correct, or control his user model.

Scenario 2 is potentially less satisfactory. It is similar to the now widespread Web personalization across diverse products such as Facebook, Yahoo, Google, and many personalized news sites. Mynews also has a model of the user, Ann. This represents her news interests and determines how Mynews selects and ranks news headlines. A scrutably personalized system aims to enable people like Jim and Ann to scrutinize their own user models to answer questions such as those in the following examples of six types of questions, in terms of our two scenarios:

(1) How did the coach determine I didn't know *undo*?
(1) How did Mynews determine I am interested in that news article?
(2) What else does the coach think I know (or don't know)?
(2) What else does Mynews think I am interested in (or not)?
(3) How can I tell the coach what I want to know (or not)?
(3) How can I tell Mynews what I am interested in (or not)?
(4) Why did the coach choose to advise me about *undo*?
(4) Why did Mynews choose that article as the top one?
(5) What would the coach do if it judged I knew things when I did not?
(5) What would Mynews do if it had errors in my interest model?
(6) What do other people I work with know about the editor?
(6) What items does Mynews give other people?

The first three question types relate primarily to the user model. The next two deal with the way that the model is interpreted and used. The last involves the models of other people. We now discuss these to illustrate our vision for scrutable user modeling.

Scrutability of the user model. To answer Question Type 1, the user needs an interface to the relevant part of his user model, to see both the *evidence source* and the *interpretation processes* determining the value of that part of the model. For Jim's case, this may be:

—*evidence source*: the editor has monitored your activity for three years;
—*interpretation process*: in all that time, you never used *undo*; from this, the coach infers you do not know about it.

Cases like Mynews are usually rather complex and so the explanation offered to Ann may look like:

—*evidence source*: Mynews tracks the headings you click on;
—*interpretation process*: it counts the words in all these articles and compares them with the words in *Greens Party Politician Misuses Travel Budget* to calculate an interest score.

A fuller explanation would allow Ann to see the actual word scores in that article and then to see how each of these was determined (in terms of a list of all the articles that contributed to the score). If a scrutable user model can enable people to find answers to Question Type 1, this is a basis for tackling the problems of *errors in user models* as well as the *invisibility problem*.

Question Type 2 calls for an overview of the whole user model for this application. Jim needs an interface showing the other parts of the model used by the editor coach. This may be large, reflecting the complexity of the editing tool (and many other software tools that are in wide use today). The model used by Mynews is likely be far larger, showing the full set of interests modeled. Scrutable user models that provide answers

to this class of question can help address aspects of the *invisibility problem* and may enable the user to identify *errors in user models*. The overview of a person's user model could enable her to realize there are useful aspects she did not even know about. This could be a basis for learning more and so could tackle the problem of *wasted user models*.

Question Type 3 relates to the issue of *control*. Once users can see their user models, and find an error or omission, they should also be able to correct it. For example, if Jim actually does know about *undo* but has never needed it, he may wish to correct his model. Of course, this could create the possibility that the user will corrupt his model. Indeed, we observed one person did this in our studies of people using a scrutiny interface for the sam editor [Kay 1995, 1999]. One may argue that this is the user's right; after all it is his model (as privacy legislation indicates). Alternatively, it could be the basis for beginning a negotiation [Bull et al. 1995]. Our approach has been to simply treat the user as one *evidence source* and to add his self-assessment to the model, then allow multiple interpretations of the available evidence [Kay 1995].

Scrutability of the personalization - use of the model. Answers to Question Type 4 involve both the user model and the application decision processes. In Jim's case, this question calls for an explanation of the coach's choice of the teaching goal, *undo*, over other things he does not know. For Jim's coach, answering this question may involve:

—the editor model as a whole (as in Question Type 2)
—an explanation of how this was used to select this teaching goal.

In Ann's case, it must explain why this article ranked higher than others:

—the Mynews model as a whole (from Question Type 2);
—Mynews ranking strategy including its model for important and current news.

This explanation would enable Ann to see why her recommendation differs over time and the ways it differs for other people. A system supporting Question Type 4 would help address the *invisibility problem* and potentially problems due to *errors in user models* in terms of the way that applications interpret and actually use the model.

Question Type 5 would enable the user to do what-if experiments, exploring how the results would differ after certain changes to the model. The point here is that if the user model is both scrutable and temporarily alterable by the user, this also gives an indirect mechanism for the user to explore aspects of a personalized application that is not itself scrutable. Answers to this class of question address the *invisibility problem*, provide a new use for the model, addressing *wasted user models* and providing a form of *control*.

Sharing user models. The sixth class of question involves seeing other people's models. This can be useful in several ways. First, it may help the user see normative models. A simple form of this is typically available in current Learning Management Systems (LMS) which can show the individual learner her place in the class, with the class average and grade distribution. This question relates to sharing the model, since those other people would need to make their models available, even if this is only in aggregate, blurred, or anonymized form (as, for example, in Berkovsky et al. [2007]). Answers to this type of question relate to the *privacy problem* and are important for enabling people to really understand their own model, so tackling the problem of *wasted user models*.

3. SCRUTABLE PERSONALIZATION CASE STUDIES

We now present three case studies that illustrate our exploration of how to tackle some of the challenges of creating scrutable user models and personalization. For each,

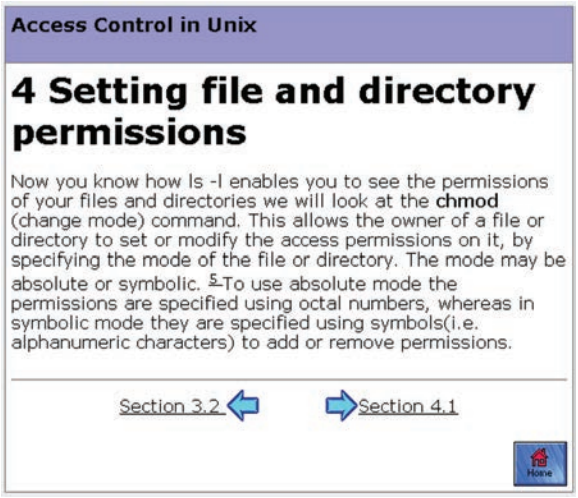

Fig. 1. Example of the way that one would typically see a personalized Web page. The user has no way to determine that there is personalization. If she did know there was personalization, there is no way to determine what has been personalized or how the user model affected the personalization.

we begin by discussing the problems it tackles. Then we explain the motivation and user view for the main application used in evaluating the system (each was built on a general-purpose framework for creating a class of scrutably personalized systems). We conclude with discussion of the evaluation of the scrutability.

3.1. Case Study: SASY (Scrutably Adaptive hypertext SYstem)

SASY is a framework to support creation of Web pages that are personalized. It tackles the *invisibility problem* by providing users with the means to understand and control the personalization. It supports a very simple but useful form of Web personalization based on selective presentation: all users see some information but other parts appear for some users and not for others. It was used in the earliest Web-based personalized systems [Kay and Kummerfeld 1994] and can support personalization of both the presentation and navigation [Brusilovsky 1996] but would be unwieldy for adaptive ordering or recommenders.

Consider the screen in Figure 1. It was part of an online tutorial system about unix file permissions [Czarkowski and Kay 2006; Czarkowski 2006]. When a user views a page like this, she has no way to know that it has been personalized at all. It is just possible that she will recall that on her first login, she answered questions like: *Do you like to have jokes included in teaching materials?* She may also notice that this page has no jokes. And she may infer that the underlying system has used her answer to the question to omit jokes from this page. Of course, this is rather unlikely, especially if the user answered the question some time ago and if there are jokes only on selected pages.

Now consider the screen in Figure 2. Note that the largest cell, at the left, has the same material as the previous screen. In addition, it has a link at the lower left: *How was this page adapted to you?* In our first attempt at supporting scrutability, this was the sole link for users to scrutinize the personalization [Czarkowski and Kay 2000]. In user studies, people simply did not find this [Czarkowski and Kay 2006]. This situation poses a dilemma for the interface designer. On the one hand, scrutiny support should be unobtrusive, so that it does not distract the user from the task at hand. On the other

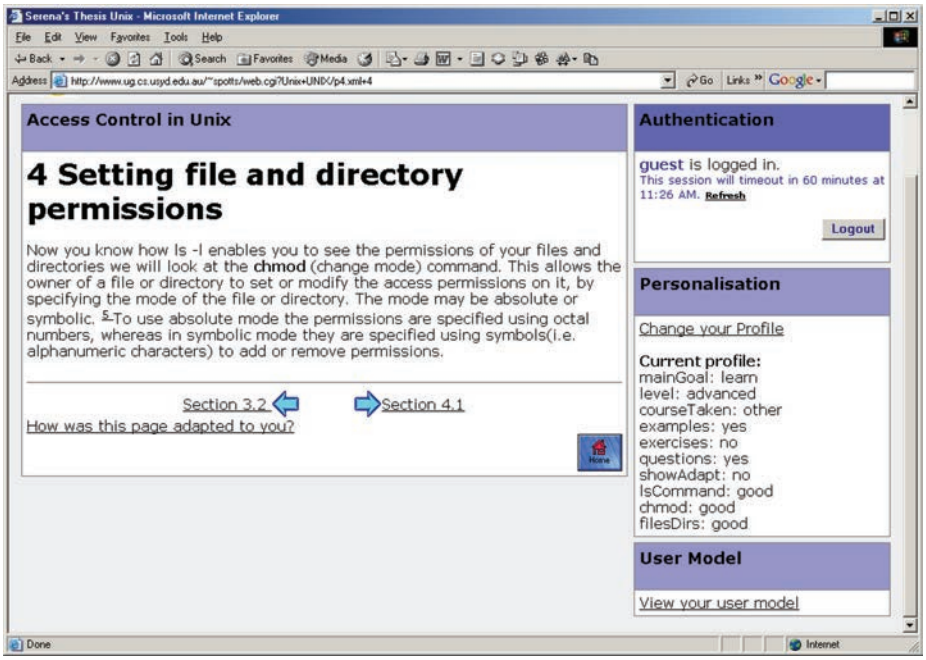

Fig. 2. Personalized Web page with clear additional indicators that there is personalization and that the user can click on the links to find more about it.

hand, it cannot be considered to really support scrutability and empower people if they cannot work out how to use it.

This is why we introduced the very noticeable and explicit information in the cells at the right. The *personalization* cell draws the user's attention to a brief form of the user model components and their values. Note that this interface presents only the current values for the components. For example, the *mainGoal* allowed the user to indicate whether he was learning this material for the first time or just revising it. The evidence for this component comes from the user's answers to the start-up questions as shown in Figure 3. If the user does not answer, no evidence is added. If he selects either of the two other answers, that causes a piece of evidence to be added to the model for *mainGoal*. At future sessions, his last answer appears and if he alters that, new evidence is added with that value. In addition, the system supports quizzes and the user's performance on these contributes to the model of the user's knowledge. The lower right cell of Figure 2 has a link to the user model.

We now consider what the user sees if he clicks on the link *How was this page adapted to you?* This transforms the page to appear as shown in Figure 4. Content included by the personalization is now highlighted in yellow and the excluded content appears, highlighted in green. In addition, mousing over either of these regions gives a popup, as shown in the figure, with an explanation of the reason for the inclusion/exclusion. In this case, it indicates that the material was excluded as the user was not at the level *basic*. (The second component in the *Personalization* cell, *level*, has the value *advanced*.

This case study illustrates scrutable personalization. As the form of personalization is quite simple, it is feasible to present the small user model on the same screen as the main content. It is also feasible to show the user the nature of the personalization. Even so, creating the scrutiny interfaces proved quite challenging. We concluded that a major reason for this is that people are unfamiliar with the notion that they can scrutinize how

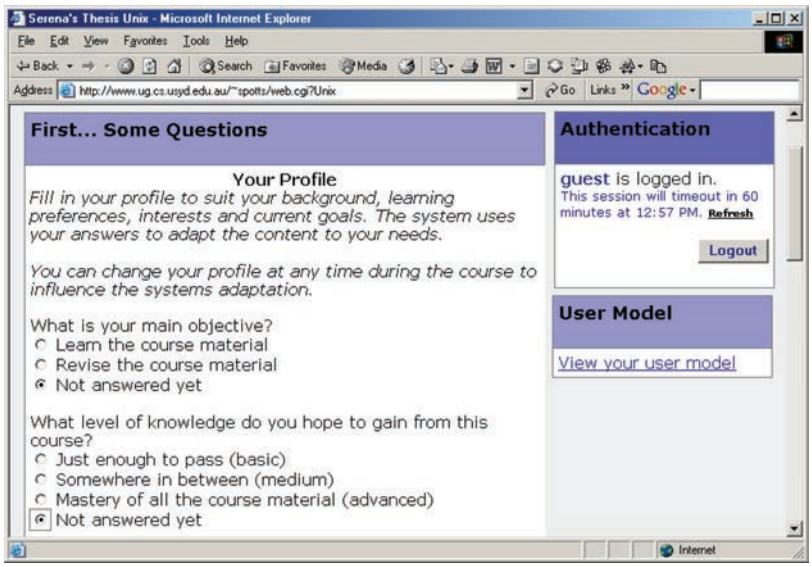

Fig. 3. This first part of the start-up questionnaire that primes the user model values at first login.

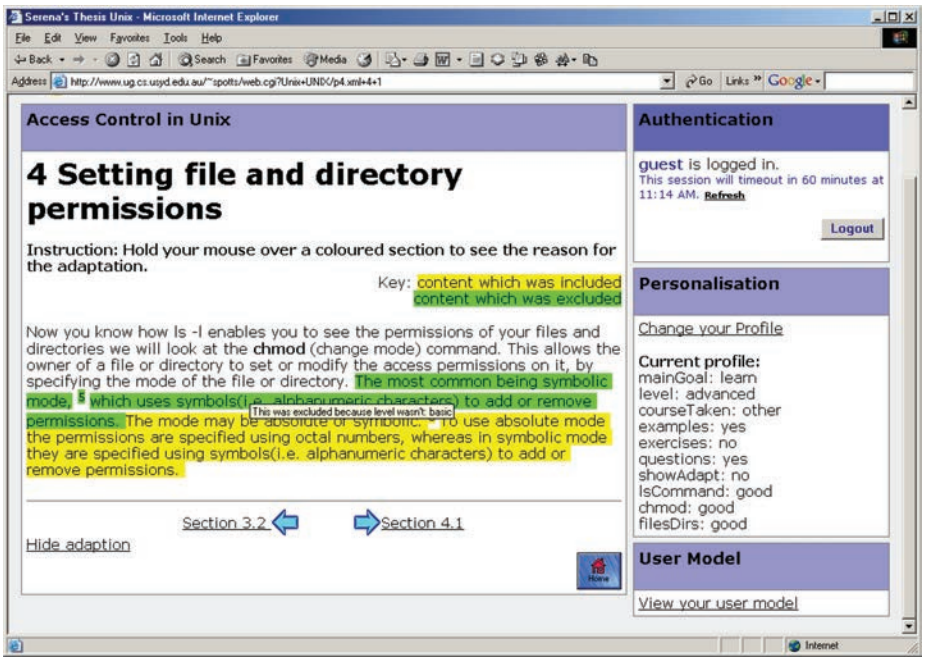

Fig. 4. Interface showing the personalization, in terms of what had been included or excluded.

personaliztion operates [Czarkowski and Kay 2006]. They ignored, or tolerated, obvious errors in the personalization, as they are obliged to do in most interfaces they use.

SASY is a framework for building scrutably adapted hypertext. It evolved iteratively as we created prototypes, then evaluated them to refine the next prototype, so gaining understanding of how to create scrutiny interfaces that people could use effectively

[Czarkowski 2006]. We created various adaptive systems including a guide to the personalization, a holiday planner, TV guide, and tutoring systems for parsing [Czarkowski and Kay 2000] and unix [Czarkowski and Kay 2006]. We used two main approaches for our laboratory trials: one asked the user to pretend to be a hypothetical user and the other asked participants to make use of the system as they wished. The first was valuable in that we defined the user's high-level goals and needs and all participants were highly comparable. The second did not have this attribute, as people could, and did, take many different paths through the personalized information space; the merit of this approach is that it is more authentic. Another possible disadvantage of the first, hypothetical user, approach is that it may have imposed additional cognitive load on participants. Both approaches were extremely valuable in our refinement of the interfaces.

In the main summative evaluation, we created a tutorial for unix file permissions and made it available for students in an HCI class where they needed this knowledge for practical work. The tutorial started with a pretest, followed by three modules, each with its own test. There was a final posttest and a questionnaire about the personalization, scrutiny support, and seeking free comments. The tutorial system was used by 105 students, with 72 completing the all aspects meaningfully. We had set defaults in the model to values that we expected to irritate users, motivating them to scrutinize the personalization. So, the system presented jokes and hints, additional advanced material, additional quiz questions, treating them as if they had no knowledge if they had even 1 incorrect answer in the pretest. At least one use of a scrutiny tool was made by 81 (77%) of students, with 13 (12%) doing 9 or more, with this spread across the tools: the profile tool; the evidence tool with most of this for the model of domain knowledge; using the personalization highlighting tool, and changing the profile. We analyzed the timing of scrutiny actions: 108 (28%) were just after completing a module (by 32 users) and most of the remaining actions were before starting a module. (Just 14 actions, 3.6%, were at other times.) We concluded that the use of the scrutiny tools was quite high, in the context of students using a tutorial system for a real learning need. We also concluded that the users were more interested in scrutinizing the model of their knowledge than the personalization of jokes, hints, advanced concepts, and extra quiz questions.

The questionnaire indicated that the users did not mind (71%) or notice (22%) the jokes; for hints, the numbers were 60% and 21%; this helps explain why only 4 altered their profiles to remove hints and 8 to remove jokes (of the 72 completing all tasks). About half, 37 (51%), agreed they could inspect what was included or removed in their personalized view (and 13 of them actually did so) and could change their model to control this (and 14 had changed their profile). On the other hand, 11 (15%) disagreed or strongly disagreed. Of these, 7 had used the profile tool to change attributes, and 3 had used the highlight tool. When asked if they were able to inspect and control the personalization, 43 (60%) agreed and 7 (10%) disagreed. We asked students who had inspected the personalization for their main reason. The responses selected were: 42% *curiosity*, 6% *I did not like how a page was personalized to me*, 6% *a belief about me was wrong*, and 6% selected *other*. Those who had not used the tools were asked the reason for this; 36% selected the response *Did not feel compelled to do so*, 31% selected *Trusted the system's default personalization*, and 4% selected *other*, with 2 commenting *to[o] much else to do!* and *didn't think it would make a difference*.

We designed the evaluation to provoke users to scrutinize, by setting what we expected would be poor defaults. Only 2 (2%) users scrutinized evidence associated with jokes, none for the hints, five for inclusion of advanced concepts, and 14% for extra quiz questions. Similarly small numbers altered their profiles; 4 (4%) to remove hints; 8 (8%) to remove jokes; 1 to remove advanced concepts; 6 to reduce the number of quiz questions. Most of the scrutiny activity was associated with the knowledge model, with

37 (35%) of the students doing so, including 11 (10%) checking our incorrect inference that they did not know a concept, when they had just correctly answered a quiz question on it with 5 changing their profile. Overall, our provocations had a small effect on user behavior. This is partly explained by the fact that the users mainly devoted their attention to the learning task at hand. (The posttest quiz showed learning gains.) Another reason that the jokes, hints, and extra material would not have been particularly irritating is that the interface clearly coded them. This is good interface design, making it easy for people to find what they want to read. One of the criticisms leveled at personalization researchers is that poor interface design may be the reason that the personalization is needed; we wanted to avoid that.

Our overall evaluation approach involved a series of lab trials and then an authentic field trial to gain deeper understanding of how people would respond to and use the scrutiny interface. An important part of the design of this study is that the tutorial system met real learning needs. In light of this, we conclude that there were quite high levels of scrutiny: 81 users (77%) scrutinized in some way, 13 (12%) scrutinized 9 or more times and used all the scrutiny tools. Most scrutiny actions (96%) were just before or right after students finished a learning module, session, pre- or posttest. Thirty-seven (35%) students changed their user model. Notably 10% of scrutiny actions were before the first pretest, indicating the scrutiny tools were noticeable and encouraged some exploration at first use. The questionnaire responses indicated that 43 users (60%) agreed it was useful to be able to inspect and control the personalization,while 6 (8%) disagreed. Broadly, this class of evaluation is important for gaining understanding of how to create frameworks, interfaces, and applications that support scrutable personalization.

3.2. Case Study: JITT (Just-In-Time Training system)

Our second case study tackles the *invisibility problem* as it has an interface to its user model, and this has links enabling the user to scrutinize details of the evidence informing it. This can avoid the problem of *wasted user models* as it enables the user to learn from the system's model of her knowledge. It provides two forms of *control*. One allows the user to select the interpretation of the user model evidence, potentially avoiding some forms of *errors in user models*. The other allows the user to select the personalization mechanisms.

The system was called JITT (Just-In-Time Training system). It was inspired by the needs of people who work within organizations that use complex workflows. JITT aimed to provide context-sensitive training materials, relevant to the current stage in a long-term task. While such systems typically work within commercial contexts, our demonstrator instance was for the process of writing examination papers in one organization. This task is very important, especially for large classes. From an institutional perspective, it is critical for assessing the effectiveness of teaching (and learning). From a student's perspective, errors in exams and unexpected, nonstandard formats can be very distressing and unfair. There are seven stages in this case: writing questions; designing the grading scheme; checks by a domain expert and by administrative staff. Typically academics do this just twice a year, and they may forget the recommended processes, making it representative of many tasks in diverse workplaces.

Figure 5 shows the main screen for the user (Judy) who is part way through the process of writing two examination papers. These are listed in the cell at the upper left. If the user clicks the link for one of these, the screen moves to a cell with its details. These provide links to the training materials for each task. The upper right cell "Important Items" is a list of links to documents that the user has identified as important in previous interactions. The other cell at the right shows the documents viewed with the most recent at the top.

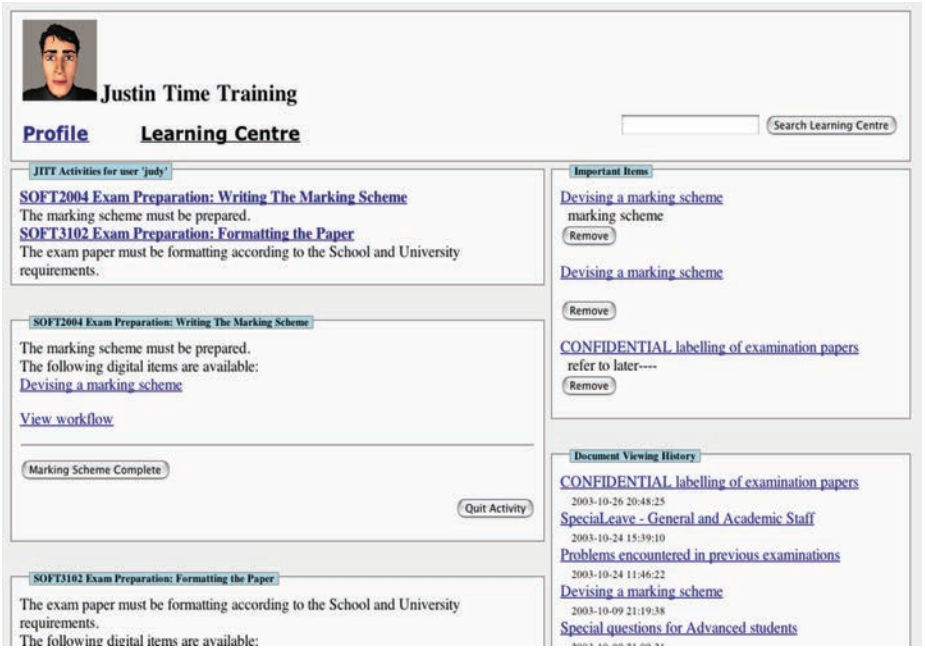

Fig. 5. Main screen of the Just-In-Time Training interface.

We can already see the two current activities for the user on the screen. Both involve writing exam papers but the user is at different stages in each. For the first of these (SOFT2004 exam), we can see what the user needs to do next: "The marking scheme must be prepared". Below this are links to the available learning materials, in this case one document. Clicking the link "View workflow" below this takes the user to the screen shown in Figure 6. This is a simple user model interface showing a graphical display of the workflow with the system's models of the user's current state in the workflow. In the underlying system, each node has an associated list of concepts that a person needs to know to do that stage of the workflow. JITT also maintains a user model that represents the user's knowledge of the set of concepts for each workflow. JITT has an algorithm which determines learning paths for the user. This begins by setting learning goals (the concepts needed for the current workflow step). For each concept that the user does not appear to *know well enough*, it finds the sets of documents that each give a "learning path" that will enable the learner to move from his current knowledge state to the goal state. In general, the workflow and the concept and document spaces can be very large. There may be many learning paths. JITT can support multiple teaching strategies which can rank these differently. The user can choose among these. For example, one may have short abstract explanations where another had longer ones with detailed examples. In the case of the exam workflow, the user model is small and there is a quite small space of training materials, all in the form of documents.

If the user clicks on the "Profile" link in Figures 5 or 6, this takes him to the page shown in Figures 7 and 8. In the top of this page (Figure 7) the user model displays the user's knowledge as one of 6 levels, with all five boxes white for cases where the user is modeled to have no knowledge of a concept, and all blue for mastery. For each component of the model, there are links to the "raw evidence" and to the "resolver explanation". The latter is a text description of the way that the evidence was interpreted to give the knowledge-level value displayed. The term *resolver* refers to the task of resolving the

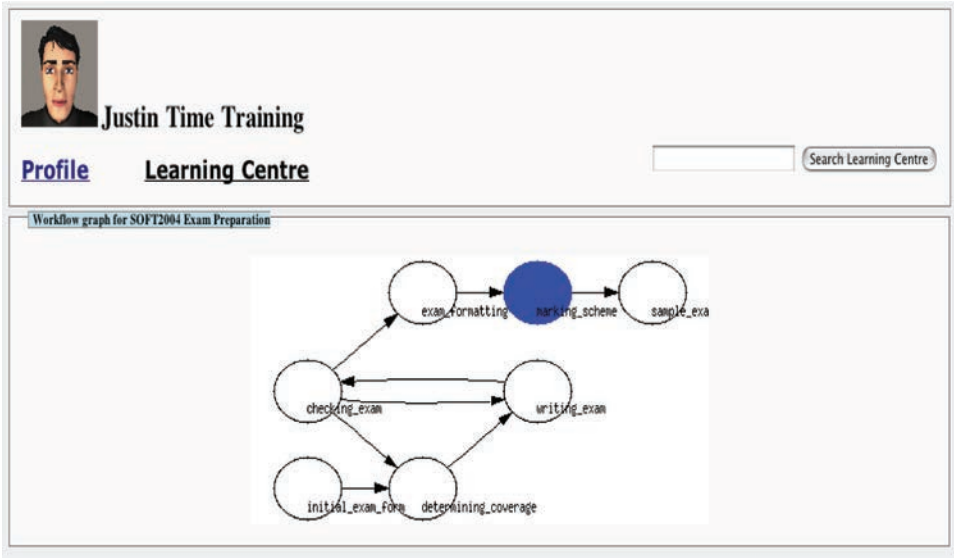

Fig. 6. Workflow display showing where the user is up to in the current instance of this task. (Labels on the nodes are a shorthand description of each stage in the workflow.)

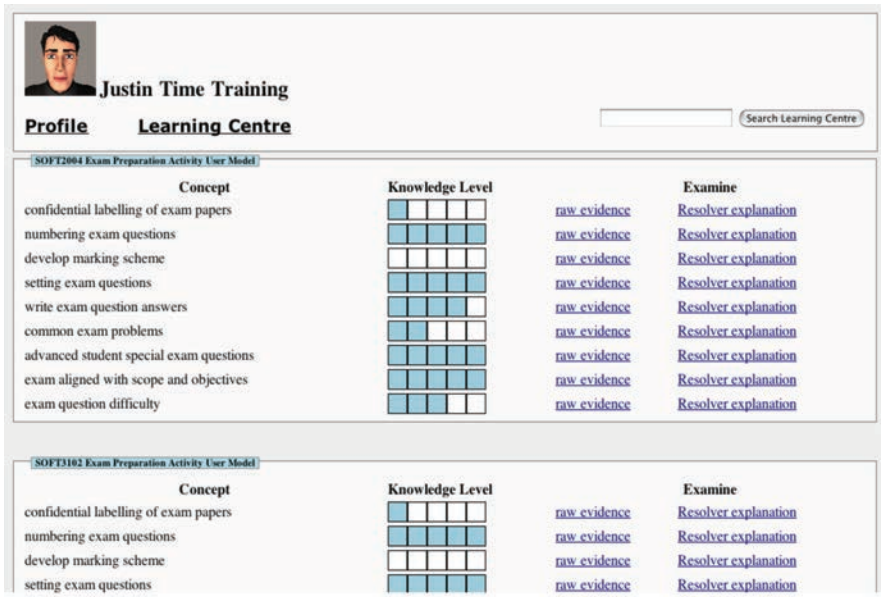

Fig. 7. Top part of the screen showing the user model.

value from a set of evidence. Figure 8 shows the system is using “Trust Most Recent”, which simply takes the value of the last piece of evidence. The user could also select the “Simple Average”. (Other options appear when the user clicks the widget.) Below this, the figure shows a cell for “Teaching Agent Selection”. This is currently the default “JITT Base”.

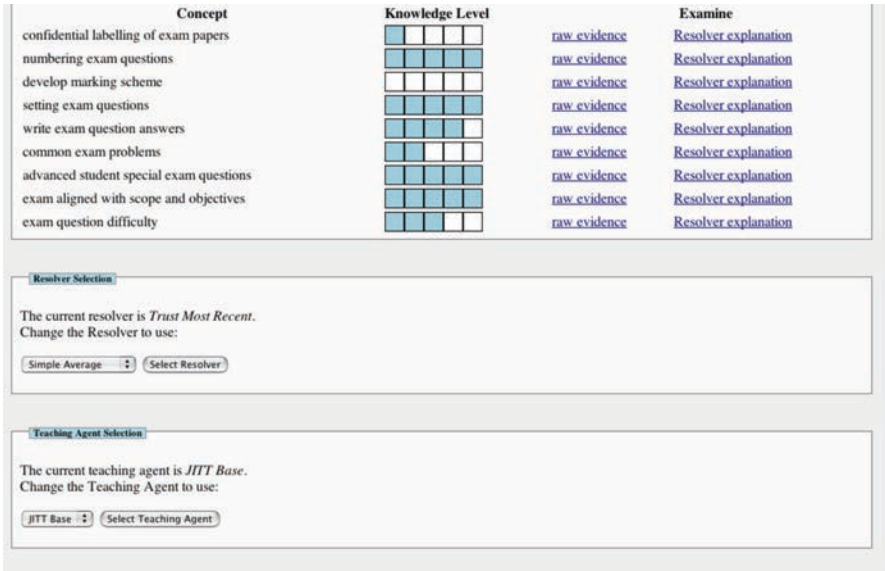

Fig. 8. Bottom part of the screen showing the user model.

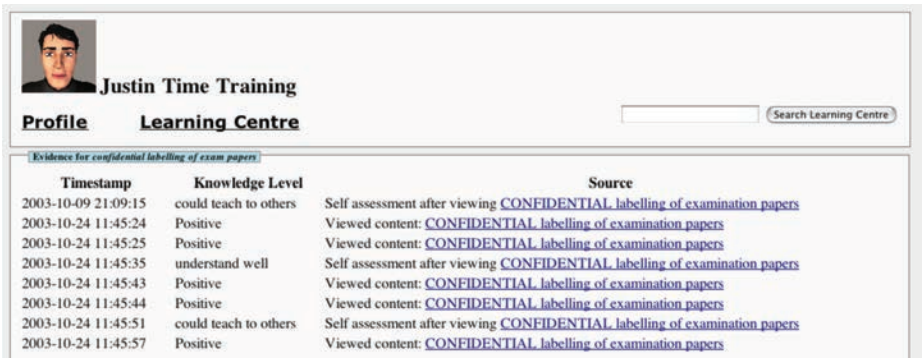

Fig. 9. Example of the evidence for the component of the model for *Confidential labeling of exam papers*.

Clicking on the "raw evidence" link in Figure 7 gives the screen in Figure 9. The top piece of evidence was a self-assessment. When the user is presented with training material, JITT asks the user to assess his own understanding of each of the concepts in the material. This is valuable from a learning perspective since it helps the user appreciate just what aspects the material was intended to teach. It is also valuable in helping the learner reflect on how much he actually feels that he has learned. A learner may well think that he has read a document carefully but not realize that he did not really take in one of the concepts listed. Equally, he may have simply decided to skim some parts of the document. This makes the self-assessment valuable both from a learning perspective and as a way for the learner to contribute to his learner model. In this case, the learner has rated himself as knowing this concept at the highest level, "could teach it to others" [Tamir 1984]. The second piece of evidence was added to the model by JITT when the user was presented with a training document. JITT's resolvers weight such evidence as weaker indications of learning than the self-assessment scores.

JITT aimed to provide a framework for personalized teaching with scrutability of the model and user control to select the evidence interpretation (resolver) and teaching strategy. To evaluate it, we designed a user study to assess:

—usability of JITT, including the scrutiny elements;
—understandability of the available choice in Teaching Agent and Resolver;
—broad user responses to the scrutability.

We conducted a think-aloud evaluation where 10 participants played the role of a new staff member writing his/her first exam. Since this is a time-consuming task, we provided a complete draft exam, with marking scheme and other associated materials for a course based on the C programming language. We asked participants to use JITT to identify any problems in the draft, according to the set policies. There were 12 problems and finding these required participants to make use of key concepts across the workflow. After completing this task, participants completed a questionnaire for background information.

Given the nature of the task, participants needed considerable programming expertise. We recruited five academic staff and five tutors (teaching assistants) of whom three had graded exams (T1,2,4). All had considerable familiarity with the examination process!

We analyzed the success and behavior of the participants in finding the 12 problems, and in using the system. The performance results indicate that participants engaged with the task, making serious use of JITT. The five tutors T1-5 found 6, 3, 10, 10, and 5 errors respectively (average 6.8) and the academics A1-5 found 5, 2, 2, 6, 6, and 5 respectively (average 4.2). The tutors did better than the academics for eight of the twelve problems with academics sometimes distracted, notably by disagreements with the policies. As one might expect, the academics sometimes relied on experience, rather than the system. Observations of participants, as they thought aloud, indicated that they could discover and understand all elements of the interface. Free-form responses to the questionnaire indicated that all agreed the system presented materials in a useful way and that JITT would be useful both when writing an exam for the first time and as a refresher later.

We now consider the scrutability and control aspects. All participants demonstrated understanding of the scrutability interfaces by using them to see their model, the evidence, and to consider the resolvers and teaching agents available. All participants were able to explain the difference between the two available teaching agents. All but one participant preferred the one providing complete information, where the personalization simply reduced the font for materials assessed as known. Participants were asked about the value of making the user model available. Table I summarizes the results, grouped by the problems we identified in the Introduction (and shown in italics).

Overall the evaluation integrated assessment of the usability and learning value of JITT along with the scrutability. This aspect of the evaluation design is important since these are integrated in any authentic use and they interact. The observations indicated that users were able to use all elements of the interface, including the scrutiny and control elements. An important caveat for these results follows from the highly technical background of the participants. Even so, JITT seemed usable and useful and the participants saw value in the scrutability of the model, with participants' free comments linking this to all five of our identified personalization problems.

3.3. Case Study: Personalized Indoor Location Information Interface

This case study tackled the *invisibility problem* in the context of a pervasive computing application. It aimed to provide personalized information about people's indoor location,

Table I. Summary of JITT Question: What Value, if Any, Did You See in Making the User Model Available?

Invisibility	
good to know what system believes	T3 T5 A4
user can see if it is accurate	T3
if model large and difficult to understand	A5
Errors in user model	
important when something goes wrong	A3
user can see if it is accurate	T3
Wasted user models	
could help identify knowledge gaps	T4 T5 A1
user can see how much the system thinks they know	T3
Privacy	
not helpful for user but helpful for supervisors	T1
Control	
allows change of resolver if they think it is more accurate for them	T3
Other very valuable	T2
not essential but useful as the system guides you	A2
unclear in this case	A4

using sensor information available without requiring people to carry special devices. This system relied on many sensors, which gave noisy and inconsistent evidence about people's location. It also made use of quite complex reasoning, based upon a personalized, semiautomatically generated ontology which modeled the user's conception of the space and her links to other people.

From the user's perspective, the system enabled people who work in a large building to determine the location of other people who work there. This had two main roles. One was as a form of "calm", ambient display that people could glance at a few times a day to gain awareness of who is around. The other goal was to enable a person to locate a particular individual. For example, suppose Alice and Bob work on different floors. They are both working late towards an impending project deadline. Alice reaches a point where she wants to speak with Bob. If he is at his desk, she will walk there. If he is the tearoom, she will call his mobile phone. However, if he is at home, she will send email; since it is late, she would not want to call him at home.

In our series of systems and interfaces called *Locator*, we tackled several technical and interface challenges. We created systems-level support for capturing sensor data [Carmichael et al. 2005; Assad et al. 2007], reasoning about available evidence to infer the user's location [Carmichael et al. 2005; Assad et al. 2007; Niu and Kay 2008] and personalizing the interface that presents the information [Niu and Kay 2010].

Figure 10 shows the version of Locator that makes use of *ontologies* for several of these aspects. It automatically built a layered ontology to help reason about the noisy, unreliable, uncertain, and inconsistent sensor evidence [Niu and Kay 2008]. It created *personalized ontologies* [Niu and Kay 2010] to personalize the information presented. The top half of the screen serves the role of the ambient display. The user has selected Level 3 of the building and this shows dots representing the location of several people. The brightness of the dot indicates the freshness of the location information, with the bright red dots being based on recent sensor data and the lighter (pink) ones being older. Below this is a detailed description of the location of these users (anonymized in the figure). This information reflects the granularity of the actual best location estimate.

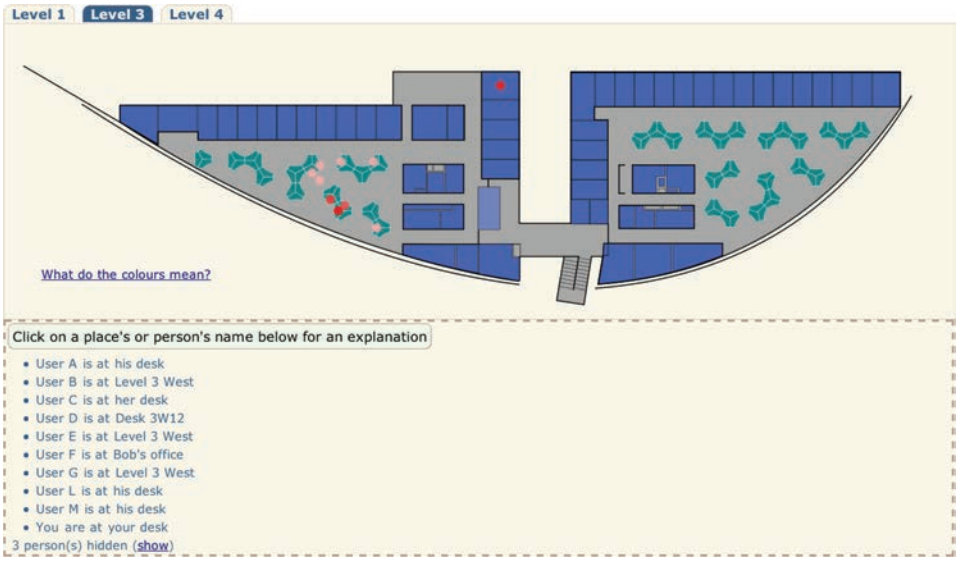

Fig. 10. Locator interface.

So, for example, User A is modeled as being at "his desk" while the information for User B is only accurate to the level of the wing. Three people have been hidden as Locator predicts that this user is not interested in their location. Locator aims to present place names that are meaningful to the user. So, for example, if the user works closely with User L, they are likely to know L's workplace desk or office. This makes it more natural to refer to that location as L's desk than use the desk identifier.

If the user clicks a person's name, Locator brings up the explanation shown in Figure 11. This enables the user to scrutinize the personalization of the description of User L's location as being at "his desk". It presents the system's reasoning. First it reports the user model value for the location of L as a particular desk (3W32). It then explains the reasoning for using "his desk": that 3W32 is L's desk and that the system infers that this user knows L's desk. It also provides the evidence it used to infer this; that L appears to be a colleague and the user seems to be familiar with the area of L's desk.

Figure 12 shows the interface as the user scrutinizes the personalization by clicking the all the "why" links to drill down to further details of the explanation. The first of these reports the evidence used to infer the location. This is from the sensor (called niu@pc-g61b-1). User L had installed this on her desktop and it sends a piece of evidence about the user's location once a minute. (Such evidence can be noisy if other people use that machine. The system had several other forms of sensors, such as sensors for Bluetooth associated with phones and laptops.) The second explanation is for the association between User L and Desk 3W32. This came from mining the "postgrad student directory". The third explanation reports the two pieces of evidence for the system's reasoning that this user and L are colleagues. First, their work areas are near each other; this was determined from the automated analysis of the building plans and was based on the reasoning that people whose work areas are close are likely to know each other. The second piece of evidence is that both the user and L are on the same internal email list. The final explanation, for familiarity with the area, is based on evidence that the user has been in this area recently.

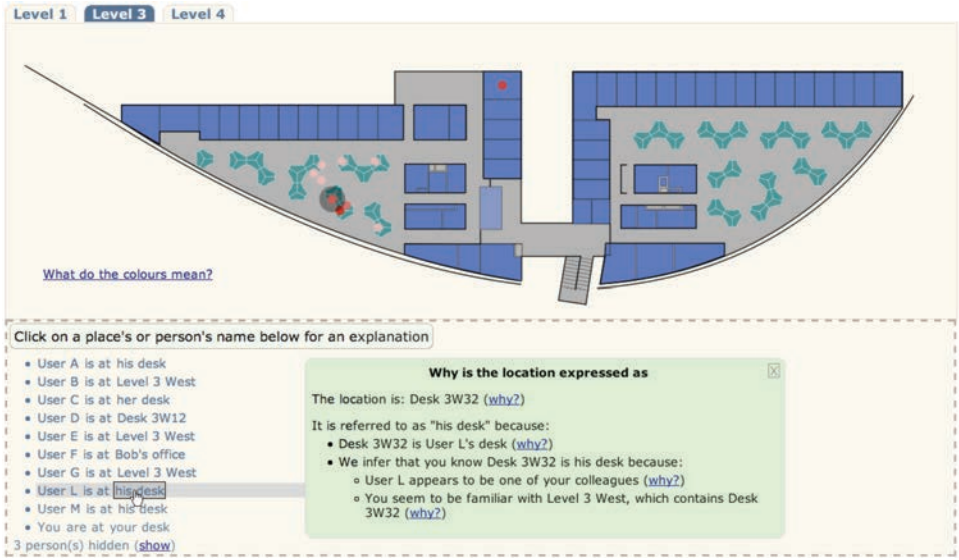

Fig. 11. Locator interface after user has decided to scrutinize further, by clicking the personalized description of a place. This presents the explanations for the personalization.

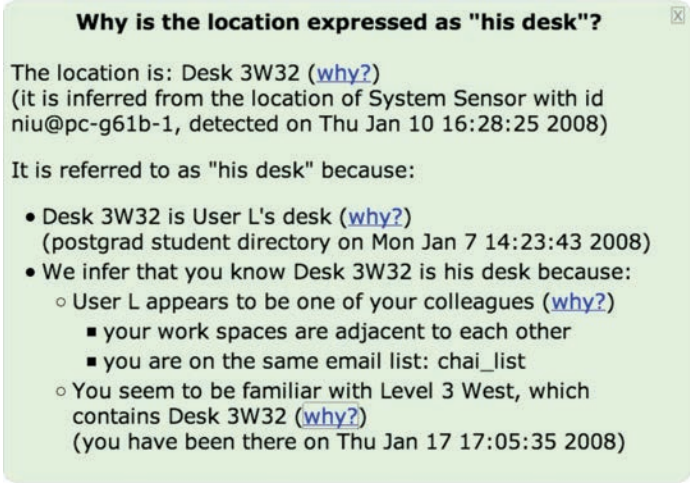

Fig. 12. Explanation from Figure 11 as the user scrutinizes the personalization, by clicking all *why?* links to show all the explanations.

The discussion to this point has glossed over the mechanisms for reasoning about the sensor data for location. We now briefly outline the ways that our ontological reasoning mechanisms support this. Figure 13 shows another version of the Locator interface. As in the interface already described, it had a similar ambient display of the building with dots showing the users. It also had a list of the users, their current location, and the last time they were detected by one of the sensors. In addition, there is an "explain" link that presents a screen with the last ten pieces of location evidence for that user.

In this case, Locator offered users the option to decide which resolver the system should use in its interpretation of the location evidence. At the upper left, we see that

Fig. 13. Another version of the Locator interface where the user could select the resolver he wished to use to interpret the sensor evidence about user's locations.

Person	Event Time	Location	Event Type
niu	1 minute ago	Level 1 West	niu-mobile
niu	1 minute ago	Desk 116D01	Login Sensor
niu	3 minutes ago	Level 1 West	niu-mobile
niu	3 minutes ago	Desk 3W32	device_niu_at_pcg61b1
niu	3 minutes ago	Level 3 West	niu-mobile
niu	3 minutes ago	Level 1 Middle	niu-mobile
niu	4 minutes ago	Room 304	niu-mobile

Fig. 14. Example of conflicting evidence about a user's location.

the system is currently using "MostRecentEvidence', a particularly simple resolver that uses just the newest location evidence for each user to conclude her location. In order to introduce the more complex resolvers, we use the examples of evidence shown in Figure 14. The ones described as "niu-mobile" are based on Bluetooth sensors that detect the user Niu's mobile phone. (When the user decided to join the system, she registered her phone, and then the system created a phone model, associating the phone's MAC-address with the user so that evidence about this phone would contribute to the location component of the user's model.) The second piece of evidence came from a login sensor, which determined that the user had logged into a particular terminal. The fourth piece of evidence is, as in the earlier example, a sensor the user installed on his desktop.

We now briefly describe the key technical and interface issues associated with the resolvers we created to interpret evidence like that in Figure 14. These make use of reasoning based on a layered ontology. The most general layer was hand-crafted and it models buildings in general; so it could be reused for a system in other buildings. The next is built automatically, by tools that analyze available resources, such as building plans, mailing lists, and other documents like the building and personnel induction manuals. While this model is not useful for other buildings, the *processes* for building it could be. The final layer is dynamic, coming from the sensor information. (For detailed descriptions of these resolvers and their evaluation, see Niu and Kay [2008] and for details of the creation of the ontologies, see Niu and Kay [2010].) These resolvers can conclude that the first three pieces of evidence are not actually in conflict, with the second being a finer-grain location within that from the first and third. The last five pieces are in genuine conflict since levels 1 and 3 are different places. We created resolvers that can deal with this by taking the majority value and combining this with the ontological model. We also created resolvers that take account of the known properties of the sensors [Carmichael et al. 2005]. For the case of Bluetooth sensors, these include: they sometimes fail to detect a device; they take up to 10 seconds to detect it; and they work through concrete floors, meaning that a level-3 sensor may detect people on either that floor or the ones above or below it. In summary, in this work, we created quite complex resolvers for interpreting noisy, uncertain, unreliable, and conflicting evidence. We did not attempt to create comprehensive explanations of these complex processes. However, we did create interfaces that enable a user to decide whether to use any of these or a far simpler one that is readily understood. Our evaluations indicate that different resolvers were more accurate for different people [Niu and Kay 2008] with simple resolvers that are easy to explain being effective for some people while the more complex ontological resolvers were more effective for others.

We designed the user study to evaluate Locator's scrutiny support [Niu and Kay 2010]. It assessed: the accuracy of the user modeling; the value of the personalized place names; understandability of the personalization explanations. For the evaluation, we created an nonadaptive version of Locator. We designed a small qualitative think-aloud study. To reduce intersubject variability, half the participants used the adaptive system first, the others used the nonadaptive one first. All completed an online postquestionnaire. With the nonadaptive Locator, participants were asked to identify locations of people and places familiar to them (assessing the personalized location labels) and name people they preferred to be hidden (assessing the accuracy of the adapted hiding). For the adaptive system, they were also asked to state *What kind of information do you think the system needs to gather in order to display "X's desk" instead of "Desk 3W32" for X's location?* and then to explore the system to assess if they were right. (To do this, they needed to use the scrutiny elements.) They were asked to determine why one of the personalized elements was displayed and why another was hidden. The nature of this system meant that the personalized display was different for each user. Participant responses to the questionnaire indicated: the explanations were understandable; they are important for personalization (although one participant rated this 5 on a 7-point Likert scale, commenting he did not care how the system achieved its personalization); and that the system explained what participants wanted to know (although for the case of hidden people, three participants wanted more detail of the reasoning in cases where the system was incorrect.) Comments pointed to a preference for more control, including setting the threshold sensitivity for displaying people (one participant) and providing a way to indicate a person should not be shown (two participants) and five of the eight participants wanted more control of the information presented, to select who to show/hide, add missing evidence, and modify

Table II. Roles for Scrutable User Models

Control of the modelling processes	**Problems addressed**
Inflow – information allowed into model (ground inferences)	Invisibility
Reasoning – inferences within model (internal inferences)	Invisibility
Use – parts of model available to an application	Privacy, Invisibility
Reuse – multiple applications using parts of model	Privacy, Invisibility, Wasted
Validity	
Correctness – enabling the user to validate/rectify model	Errors
Re-purposing	
Navigation – user model as overview of large information spaces	Invisibility, Wasted
Influence – based on explanation of personalisation	Invisibility, Wasted
Learning – user models as *mirrors*	Wasted
Usability – personal, comparative, long-term usability	Wasted
Programmer accountability – programmer mindfulness of user	Invisibility

incorrect system beliefs. Four participants expressly valued being able to discover and scrutinize the reasons for hidden information. Overall, all participants were able to use the scrutiny interface to do the four tasks involving finding people and then explaining how the various aspects of the personalization worked. The questionnaire showed participants found the explanations understandable, gave the information they wanted, and are important. The nature of the user modeling meant that participants had elected to use one of our Locator interfaces (and so had a user model). This limits the generalizability of the results. Nonetheless, the work points to the feasibility of providing scrutable interfaces that are usable and likely to be valued by users.

4. ROLES FOR SCRUTABLE USER MODELS

To this point, the article has introduced our vision for scrutable personalization with scenarios and case studies. We now move to the general roles for scrutability, summarized in Table II. It shows how these relate to the problems identified in the Introduction using the following abbreviations, Privacy for the *privacy problem*; Invisibility for the *invisibility problem*; Errors for *errors in user models*; Wasted for *wasted user models*; and as *the control problem* is linked to all of these, we omit it.

Control of the modeling processes. Privacy of personal data means that people should be able to restrict its use as they wish [Palen and Dourish 2003]. This makes it tightly linked to issues of control. If a user can scrutinize her own model to gain an understanding of it and the associated processes, this can be the starting point for controlling these. There is an increasing body of work on providing user control in personalized systems. For example, in a user study for an adaptive news system [Ahn et al. 2007], users liked the control available when the user models (or profiles) were open to them. Similarly Paramythis, Weibelzahl, and Masthoff identify scrutability as one of the criteria on which to evaluate personalized systems [Paramythis et al. 2010]. Scrutability has important roles for control associated with user model interoperability [Carmagnola et al. 2011]. Table II has four rows for the main aspects of control of the modeling processes.

—*Inflow* involves access to details such as the source of each piece of information in the model and the time the information was added.
—*Reasoning* refers to the processes used to reason from the raw information. Some of this may be very simple. For example, in Figure 15, the key at the bottom of the figure shows a mapping between the number of Google searches each day and the color saturation in the display. In many cases, the user model inferences are far more complex than this. Then it is more challenging to create interfaces to support

Fig. 15. Simple example of a user model for navigation – a model of user's daily Google search activity. In this and the next user model interface, the more intense the color, the more activity on that day. By clicking a cell, the user navigates to associated information.

scrutiny of the inference processes. For example, in a learning context based on a Bayesian user model [Zapata-Rivera and Greer 2004], learners could explore a visual form of the model graph, showing links between these and the size of each node reflecting its value.

—*Use* involves enabling the user to see all the uses (and potential uses) of the user model. This is the foundation for the user to control what information may flow out of the model. So it is a basis for controling the privacy of the model. It means the user can see which parts of the model are available to a particular application and how those are used within the application. This is an essential foundation for controling the ways that the model is used in practice. We saw this in the three case studies.

—*Reuse* refers to use of the same parts of the model by different applications. Many personalized applications have their own user model, and this is not available to any other application. Yet user models may be useful in multiple contexts. Scrutability and user control can facilitate such reuse.

Validity. One obvious role for scrutability is to enable the user to find and then address errors in user models. In fact it is so obvious that it scarcely needs discussion; it is an essential aspect of user control. This role for user models was recognized early. For example, Csinger described his video customization system [Csinger 1995] as scrutable as it showed the user parts of the user model so she could correct these. Similarly, the Doppelganger user modeling shell [Orwant 1994] and applications built upon it used a range of automated mechanisms to acquire information about the user. Orwant argued for user access to the model to ensure its accuracy and created several interfaces to display various models. Much of the work on open learner models, discussed shortly, gives the learner the opportunity to correct or challenge the system's model of him.

Repurposing user models. Table II identifies five roles that scrutable user models could play, by using electronic personal data, the problem of *wasted user models*. People are increasingly leaving digital footprints as they interact with systems. That data can be transformed into scrutable user models that have important uses that go beyond personalization.

Navigation refers to the potential for the user model to serve as an interface to an information space. An elegant example of this appeared in ELM-ART [Weber and Brusilovsky 2001], a system which teaches about programming in LISP. The

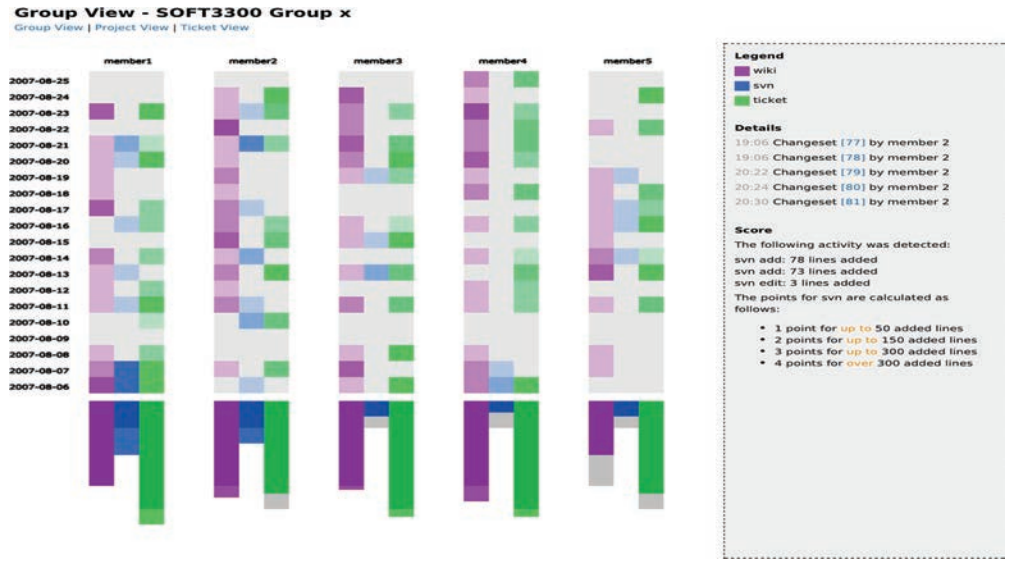

Fig. 16. User model for navigation of a complex information space – the Narcissus user model navigates a large group-information-spaces of integrated wiki, issue tracker and source-code control system, showing daily activity on these.

"course-map" for the system was rather like a contents page in a conventional book. However, it actually showed the underlying user model. It color-coded each entry, using a traffic light metaphor. It used green for the aspects the learner was modeled as ready to learn. Red indicated those where the learner lacked the required preknowledge. Black marked those already mastered. We can see a quite similar notion in Figures 15 and 16. The first is a the calendar visualization of a person's search activity on each day. Clicking the day presents details of that activity. This is a visualization of the user's Google search activity. In such interfaces, a scrutable user model can facilitate navigation of a large information space; at the same time, navigation enables the user to see her user model.

In an example that provides greater user control, Narcissus [Upton and Kay 2009] is a visualization of a model of user activity on a group collaboration platform[6] with integrated tools: a wiki; version control system; task allocation and tracking (often called issue tracking). An example screen is shown in Figure 16. Narcissus shows each user's activity, on each day of the period requested, for each of these tools in terms of the color intensity. The figure shows models for 5 users. For each, the three parts correspond to the three tools. Clicking on any square presents a list of the links to that activity (shown at the right of the screen) and clicking on any of these takes the user to the actual content, for example, the actual diff-display showing just which lines the user added to the wiki. In this case, the user can control the level of activity associated with each color intensity level. This helps users navigate what can become a very large and complex information space.

Influence refers to the potential for scrutability to serve as a positive influence on people. Research has shown that people are more likely to accept personalized recommendations that are accompanied by an explanation [Herlocker et al. 2000; Tintarev and Masthoff 2007; Cramer et al. 2008]. This seems to link to issues of trust, as well as understanding and control. It constitutes an additional role for scrutability.

[6]The platform is trac: http://trac.edgewall.org/.

Learning. The role of scrutable user models for learning relates to the potential of a user model to operate as a *mirror*, showing the user a view of himself for the aspect(s) modeled [Kay 1997]. The SMILI framework characterizes the purposes and diverse nature of such models that "invite the learner in" [Bull and Kay 2007]. Notably Bull has done extensive research on open learner models, their uses, and varied interfaces. A selection from this diverse and substantial body of work is: negotiated models where the learner could challenge the system about parts of her user model and she was then invited to demonstrate her level of knowledge [Bull et al. 1995]; a mobile phone interface to support learning on-the-go so learners could share and compare models [Bull and Mabbott 2006] as a starting point for more learning; exploration of sharing of learner models [Bull et al. 2007]; creation of different interfaces for children and parents so that parents could support their children with math homework [Lee and Bull 2008]; and helping university students understand their long-term learning [Bull and Gardner 2009]. Mitrovic found students learned more in an online course for SQL when they had open learner models, and there was a stronger effect for weaker students [Mitrovic and Martin 2006]. Dimitrova demonstrated the power of a more complex form of interactive model as a learning aid [Dimitrova 2003]. John Self identified various forms of openness that can enhance learning [Self 1999]. OLMs offer the potential for improved support for several metacognitive processes, such as reflection, self-knowledge, self-monitoring, and planning [Bull and Kay 2008].

Such learning, based upon reflection on a suitable presentation of a model of personal data, can go beyond the formal classroom setting. This has been described in the following terms: "Almost imperceptibly, numbers are infiltrating the last redoubts of the personal. Sleep, exercise, sex, food, mood, location, alertness, productivity, even spiritual well-being are being tracked and measured, shared and displayed" [Wolf 2010]. In a recent critique of lifelogging, Sellen identified five key benefits for memory that such capture technologies offer: recollecting, reminiscing, retrieving, reflecting, remembering intentions [Sellen and Whittaker 2010]. While lifelogging differs from user modeling, it shares this potential to support these forms of learning and augmenting cognition. The notion of *personal informatics* [Li et al. 2010] is similarly based on the collection of personal information in a form that can support reflection, to achieve self-knowledge.

Personal usability. Usability. Typical HCI usability measures assess whether users can complete their tasks successfully and recover from errors, based on analysis (in discount usability evaluation methods such as cognitive walkthrough or heuristic evaluation) and studies (such as think-alouds and field trials). Monitoring actual use, over the long term, in the field, to build user models can provide new forms of usability assessment [Kay and Thomas 1995; Hilbert and Redmiles 2000]. Scrutability is key to several aspects of this. *Personal Long-Term Usability*, based on an interface to a scrutable user model can enable a person to answer questions like: "What is my pattern of use of this tool?" and "What is there to know about this tool, compared with the way I have used it?" This can be a foundation for individual learning by navigating the scrutable user model [Cook and Kay 1994; Cook et al. 1995]. If others are willing to make their models available, a scrutable model can support *Comparative Long-Term Usability* answering questions like, "How does my model compare with that of others like me?" Scrutability can support *Use-based Recommendation*. For example, Linton and Schaefer [2000] tracked use of Microsoft Word to build user models and used comparative models to drive recommendations of things that people may find useful to learn. The reasoning was that a person who never used a facility is more likely to benefit from learning about it if it is heavily used by people with a similar job role. A suitable form of scrutable user model could make this information available to people

Table III. Overview of Principles for Scrutable User Modeling and Personalization

Parallel design	Design of representation and interfaces in parallel
First class citizen	User models exist outside a single application
Context-based interpretation	Each application and context may interpret the user model differently
'Client-side'	Personal digital infrastructures redefine the notion of client-side

as a basis for planning their learning. Of course, these forms of usability differ from the classic HCI view of usability because they may lack the important link to the user's intention and task. However, classic usability, with its focus on the generic user, does not give the user understanding of the usability of a tool for him as an individual.

Programmer accountability and awareness of the user. In our earliest work in designing scrutable user models for people's learning of a text editor [Kay 1990, 1995; Kay and Thomas 1995], the goal of scrutability made us mindful of the user who would view the model. So, for example, we departed from the typical distinction between knowledge and misconceptions. Instead, we defined the user model in terms of knowledge, for those aspects the system designer considered correct. We chose to use the term *belief* to model aspects where the user appeared to have a different approach from the designer. For example, we modeled whether people quit the editor by killing the window; we considered the quit command was a better way to quit (as it gives potentially valuable messages, including warnings about unsaved edit buffers). In time, we learned that our users did not read or understand such messages. So, based on their context, understanding and needs, their belief that it was appropriate to use the faster kill-window quit was reasonable. Similarly, we considered the user even as we chose names for parts of the model. Broadly, we were mindful of the user as we considered each design decision about the user model. Such awareness is valuable in helping the system designer to maintain a focus on the user's view and need.

5. PRINCIPLES FOR CREATING SCRUTABLE USER MODELS

The last section discussed *why* scrutable user models are valuable, in terms of the roles that they can serve. Now we describe *how* we can achieve scrutability in user modeling, starting with the broad set of principles listed in Table III. These come from the analysis that underpinned even the first of our work [Kay 1999], and they draw on the work of others, particularly work on user modeling shells and frameworks [Kobsa 2001; Kobsa and Fink 2006] as well as our own experience outlined in Section 7.

Parallel design refers to the need for the designers of personalized systems to consider both the interface issues and the user model representation and reasoning, together, from the beginning of the design process. This is essential if the user is to be able to scrutinize the underlying reasoning of the system at a high level of fidelity. By this we mean that the user should be able to see the actual details of the system reasoning and of the user model. Suppose a particular representation for reasoning is chosen and associated scrutiny interfaces created and it seems that people simply cannot use these interfaces effectively. It will be necessary to revisit both the choice of representation and the design of the interface. Broadly, this principle is in line with the considerable body of research in providing explanations in intelligent systems. (See, for example, the earliest expert systems that explained their reasoning in terms of the chain of rules used [Clancey 1983] to more recent work in conjunction with recommenders [Glass et al. 2008], architectures for explainable AI [Core et al. 2006], and the engineering of service-oriented adaptation [Koidl and Conlan 2008].) That work confirms that some representation and reasoning approaches are easier for people to understand. These are likely to be more amenable to scrutable user modeling when suitable interfaces are created. For other representations, it may be infeasible to support scrutability beyond a high-level description.

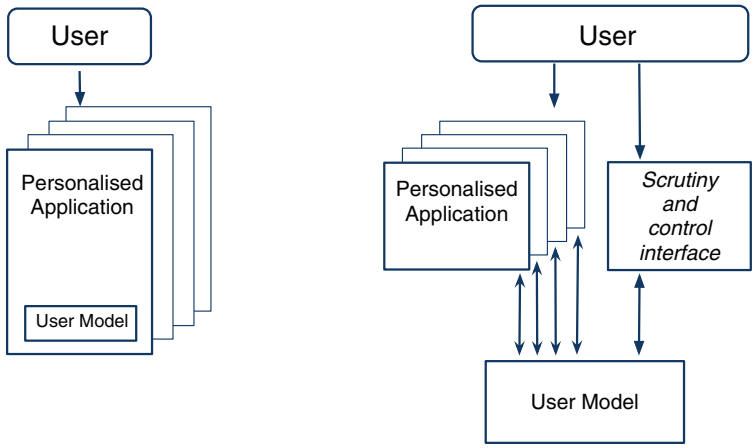

Fig. 17. Summary of the user's view of the transformation of user models to a form that users control. The left figure shows the current situation with each application holding its own user model. Our vision is shown at the right: in this, the user rather than the application owns the user model and there is an interface that enables the user to scrutinize and control his user model.

First-class citizen. This refers to the need to shift from the dominant existing paradigm where each application has its own user model, designed for its sole use. Figure 17 shows the transformation we propose. At the left, it shows the current situation: a user interacts with many personalized applications, each with its own user model. Each user model is locked away within its application. Our vision, illustrated at the right of the figure, has two key differences. First, the user model is not trapped within any one application. Rather it is a separate entity that the user controls. Second, there is an interface that enables the user to scrutinize his user model to answer questions like those in our scenarios. Making the user model a first-class citizen, under the user's control, can facilitate reuse of the model.

Context-based interpretation. This refers to the challenge of enabling a user to determine how her model is used, or will be used, in practice. Even a simple user model may be used in different ways, depending on the context or application. This is important for supporting scrutability, since a user is likely to need to understand her user model as it operates in a particular context. This means the user needs only to scrutinize the small part of the model that is relevant to that context. It is also easier to create effective interfaces that are tightly coupled to interpretation in that context. This is especially so if the user model is interpreted differently in different contexts. We initially tried to create more general scrutiny interfaces [Cook and Kay 1994; Kay 1995, 1999]. For example, these showed the user as *knowing undo* if she had told the system that she knew it. However, in such interfaces, there was no way the user could see that our coaching system interpreted the user model differently; it concluded that a person did not know an aspect if she never *used* it. In subsequent work, we have linked the scrutiny interface to the context of use. Often this is the particular application, or the current context within it, as in the first case study, where SASY showed just the parts of the model relevant to the personalization on the current page.

"Client-side" control. We now consider what appears to be an emerging notion of "client-side" control. Previously, client-side referred to personalization where the user's model was stored on the user's own computer, rather than on a remote server. This has advantages for privacy [Kobsa 2007] but at a cost; the client-side model cannot make use of other people's models as needed for collaborative recommenders and service providers may have a business model that is incompatible with it. The notion of

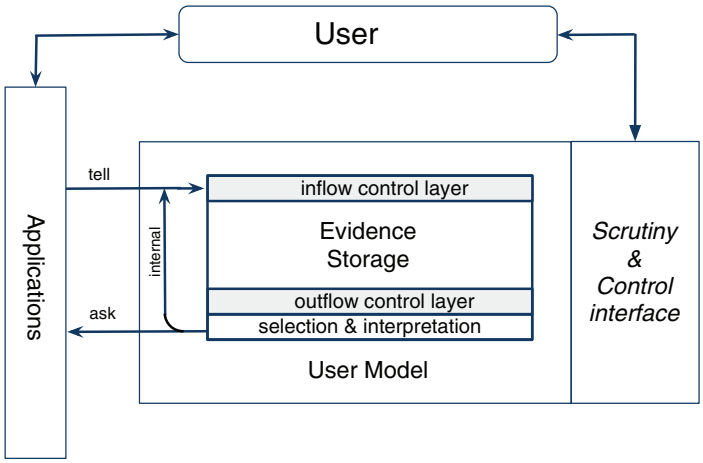

Fig. 18. Overview of architecture for evidence-based user model. The user interacts with conventional applications, or is tracked by sensors. These may "tell" evidence to the user model with an inflow control layer of the model determining what evidence is allowed into the main evidence store. Applications may also "ask" for information about the user. The result is determined by controlling the evidence to this application as well as its choice of the allowed mechanisms for selecting parts of the model and interpreting the evidence. The model can use "internal" inference to create new evidence. At the right is the scrutiny and control interface.

client-side user modeling can be viewed as changing, as indicated in Figure 17 where the user model can be seen as a separate entity, and client-side control means that this is under the user's control. As the user's personal digital ecosystem involves increasing numbers of devices as well as multiple stores of personal information, client-side storage might be considered to include all the places where the user has control over the information store. This is relatively straightforward in the case of a personal desktop computer since client-side user models would be stored on that machine. However, rich network connectivity means that it can be very convenient to have personal information stores that are on many different devices (such as phones and laptops) and also in the "cloud". In terms of commercial pragmatics, this broader view of client-side or client-controled user modeling may be compatible with pragmatic issues described before. The shift needed from current practice is that the user should have access to and control over his user model, regardless of where it is stored. In terms of scrutability, this means that the user needs to be able to find all the relevant parts of his user model, and then explore them. Many major Web-based services that store personal information already provide an API to that store. This makes it feasible for programmers (but not ordinary users) to create tools that access such user models and provide a comprehensive scrutiny interface that incorporates parts of the model, wherever they are stored. These tools could also copy the model to storage that is more directly under the user's control. The user model as first-class citizen, controlled by the user rather than by particular applications, may constitute a new form of client-side user model.

6. ARCHITECTURE FOR EVIDENCE-BASED USER MODELING

We now move further into the system builder's tasks and *how* to create scrutable user models. We argue for the high-level architecture and representation illustrated in Figure 18 (refining the right side of Figure 17). This architecture shows the user, at the top, can interact with applications and also use a scrutiny and control interface. It emphasizes management of information about the user in a similar manner to that advocated for privacy [Spiekermann and Cranor 2009].

When an application operates as an *evidence source*, it "tells" evidence to the model, as indicated by the arrow, labeled with *tell* on the figure. Evidence may take many forms. For example, a GPS location sensor may send the user's current location as a pair of values giving fine-grained latitude and longitude. A wireless bathroom scale may send the user's weight in kilograms. An editing application may send details of each command used. And, taking an example from the widely used Web applications, a click on the advertisement for a book may send evidence indicating interest in it.

When the user model receives the tell, its inflow control layer assesses whether the application is allowed to add evidence to the model. This is shown in the figure as a shaded block to indicate that it operates as a selection barrier, permitting the addition of evidence from allowed sources and blocking others. This is becoming an increasingly important element of emerging pervasive user modeling, based on automated collection of information by sensors [Langheinrich 2002; Spiekermann 2008].

Similarly, the arrow labeled *ask* operates when an application "asks" the user model for information about the user. An *outflow control layer* determines what evidence, and what form of it, is allowed for this application. This layer is also shaded because it can restrict the information allowed to the application asking. For example, suppose an application asks for the user's location and that there is a series of values from a GPS evidence source. At this stage, the available evidence may be filtered. For example, the user may want some applications, and some people, restricted to location values from the last day, within business hours and the location values may be translated to coarse-grained values. This mechanism is to address the *privacy problem*.

The figure shows a second layer controling outflowing values. This is not shaded as applications can decide which of the mechanisms they need to use here. This enables the application to *select* parts of the model and the particular interpretation tool. For example, in the case of location, there may be a choice of a symbolic location, such as {"at work", "not at work"} or values of longitude and latitude.

We call this evidence-based approach *Accretion-Resolution* (AR) [Kay 1990, 1995, 2000a]. The evidence from allowed evidence sources *accretes*, meaning that it is simply added to the model. Later, when a value is needed by an application, this value is *resolved* by interpreting the allowed evidence. In the case of noisy, unreliable, and uncertain evidence, as is common in many areas of personalization, the resolution process may be challenging (as, for example, in the Locator case study which used ontologies to improve reasoning about the user's location and the people to display). AR supports the principle of *context-based interpretation* identified in the last section.

There is an additional flow, marked *internal* in the figure. This uses inference to create new evidence about the user. Such sources of *internal evidence* should be distinguished from *external evidence* (also called *ground* inferences since they are the foundation for all internal inference). There are risks here for privacy and user control; when external, ground evidence informs internal inference and evidence, the latter should carry the same access controls.

6.1. Design of Evidence Representation

There are many ways that one could implement the core ideas of this architecture in a user modeling system and associated personalized applications. As part of our *parallel design* of the representation and scrutability interfaces, we have refined the general AR approach, requiring that each piece of evidence has at least the following.

—There should be a timestamp indicating when the evidence was added to the model (under control of the model) both to enable reasoning based on this timestamp and to support historic scrutiny.

—There is to be an optional timestamp provided by the source (for example, to indicate the time the application actually collected the evidence) for the same broad reasons as the system timestamp.
—There should be the user model value, for example, *true* if the user indicates interest in a news article. In general, different sources may contribute different types of values, such as booleans, certainty scores, or symbolic values.
—There should be the source identifier, useful in three ways: for the *inflow* control, allowing only registered and authenticated sources to contribute evidence; for reasoning based on the sources, for example, taking account of their reliability; and for scrutability since this should be available in explanations of the evidence and associated reasoning.
—There should be evidence type; our analysis of interaction led to the following important distinctions [Kay 1995]:
  —explicit ground inferences: come directly from a person, for example, when she explicitly answers a question about her preferences or knowledge;
  —implicit ground inferences: are based on monitoring, logging, and other forms of observation;
  —ex-machine ground inferences: track what the machine (actually an application) has told the user, so the system can avoid repetition and assess the success of its actions;
  —stereotype internal inferences: for statistically based inferences that are so important in user modeling [Rich 1983, 1979, 2000b];
  —inferred internal inferences: based on other forms of inference within the model, typically based on knowledge bases such as those captured in Bayesian networks.
  The evidence type can be used in explanations to the user and can be combined with the source identifier, for a lightweight form of *interpretation* of evidence (Figure 18).

There has been work that has used representations with some of these properties. For example, CUMULATE is a server which accepts data from learning systems [Yudelson et al. 2007]. Zapata-Rivera et al. [2007] used "evidentiary arguments" in different forms for different people in designing "active reports" for open learner models. An evidence-based approach was used to tackle semantic heterogeneity for interoperability in user modeling [Carmagnola and Dimitrova 2008]. Data fusion [Bleiholder and Naumann 2009], an approach that has much in common with accretion-resolution, has been described as "conflict-ignoring strategies".

6.2. Representation of the User Model

Building from this abstract architecture, we now argue for a set of principles to guide the design of a representation to support scrutable user model reasoning. Table IV summarizes these.

Trade-off refers to the the two principles, *simplicity* and adequacy, that may well be in conflict and the role of partitioning to address this trade-off. The simplicity principle is motivated by the need to create interfaces that enable the user to scrutinize her user model so that she can understand it. It seems wise to start with the simplest representation that can still support the forms of reasoning needed. The *adequacy* principle acknowledges the need for sufficient power within the representation to model the user effectively and to support reasoning as needed by the personalized applications. The *partition* principle allows the design to have some parts of the user model that use simpler reasoning which is amenable to full scrutability. Other parts may use more complex reasoning for which it is difficult, or impractical, to create a comprehensive scrutiny interface. For example, much learner modeling uses a very simple overlay model (mapping the learner's knowledge against the system's model).

Table IV. Representation Principles for Scrutable User Modeling and Personalization

Trade-off	
Simplicity	The representation should be as simple as needed for scrutability
Adequacy	It support the reasoning needed for personalisation
Partition	Complex parts of the representation should be decoupled
Ontology	
Namespaces	It needs sets of context-dependent namespaces
Semantics	The model must be able to explain the meaning of each element modelled
Accountability	
External evidence	Keep the external evidence in original form
Interpreting evidence	
Inconsistency	Model may maintain evidence that is contradictory
Multiple interpretations	Each application may interpret evidence differently
Late binding	Interpret evidence only when needed

It is relatively straightforward to create scrutiny interfaces for these. However, some parts of the reasoning may call for a more complex representation, such as a Bayesian network, or one based upon sophisticated real-time machine learning. There is ongoing research in generating explanations for a range of reasoning mechanisms, for example Lim and Dey [2010]. This principle proposes that these different mechanisms be kept separate so that the user could decide to disable parts that are not scrutable. This would match the observation that different people have different preferences in terms of control [Jameson and Schwarzkopf 2006]. It may be particularly important for pervasive computing applications where the convenience of personalization may be valuable enough for many users to relinquish some control [Barkhuus and Dey 2003]. If inscrutable and uncontrolled aspects are partitioned, systems can be created to enable the user to determine whether she wants to use these, or just the scrutable parts. A similar case has been made for design that ensures the user can understand intelligent systems, to make parts of them transparent and predictable [Hook 2000].

Ontology refers to the aspects associated with the semantics of the model. The *namespaces* principle calls for partitioning the model into parts which each have their own ontology. This means that the concept can have different meanings in different parts of the model. The *semantics* principle requires that a scrutability interface should be able to explain what is modeled about the user. In Scenario 1, this means the system should be able to explain the meaning of the part of the model for the user's knowledge of the *undo* command. This goal motivated our work to create a scrutable lightweight ontology based on analysis of available online dictionaries [Apted and Kay 2004]. We used this to generate the ontology of a user model and for visualizing it [Apted et al. 2003]. This meant that we could automatically generate explanations of the meanings of components of the model; this could make use of the dictionary definitions, already written for people to learn about the meaning of the concepts. In addition, we could create a visualization of the relationships between concepts, and combine this with dictionary definitions and a description of the ontological reasoning [Kay and Lum 2005]. There may also be a need for personalization of the explanations provided for the ontology. For example, we explained elements of the sam text editor differently, depending upon the user's modeled knowledge and expertise [Cook and Kay 1994].

Accountability calls for the representation to maintain all external evidence provided to the model. By contrast, many systems simply keep a single value for each component modeled, altering its value with each new piece of evidence, and discarding the details of the actual evidence. By keeping the actual evidence, the system can explain the modeling processes. For example, the user may be surprised that a system operates differently today from the way she recall that it operated last week. So she may wish

Table V. Interface Principles For Scrutable User Modeling and Personalization

Overview	User can gain broad understanding of their user model and the parts of model used by a particular application
Ontology	The interface should be able to explain each aspect modelled
Scrutiny	User can scrutinise all details of the user model and a particular application's use of the model
Inflow	Determine what goes into model
Give value	User can volunteer a value for a component
Outflow	Enable user to determine parts accessible to a particular application
Outflow process	Enable user to see how model is interpreted by a particular context
Use	Enable user to see how model is used in particular context

to see the value of the model as it was last week as well as now. In the case of our editor coach of Scenario 1, evidence came from analyzing logs of use. Since evidence from internal reasoning might be regenerated, it may not be necessary to keep that.

Interpreting evidence involves the reasoning that the system performs, based on the available evidence. The *inconsistency* principle reflects the fact that people may have inconsistent beliefs and that the system may have inconsistent information about the user. For example, this may be due to lack of context within the model, such as when the user likes to watch certain movies with children and has different preferences for movies to see in the company of other adults. There may be inconsistent evidence because of the nature of the aspect modeled, especially in learning contexts [Kono et al. 1992]. For example, a learner may do a series of tasks that measure his knowledge of a certain concept. He may make correct guesses when he actually does not know the concept. He may make slips when he knows it. A typical learning process might start with many incorrect answers, but some correct ones and, as the learner reaches higher levels of mastery, he may start to consistently give correct answers (bar slips). The *multiple interpretations* principle relates to the context-based interpretation principle and the need for different interpretations of the same evidence about the user, depending upon the application and other aspects of the context. *Late binding* refers to delaying the interpretation of evidence until it is needed. Just as the actual evidence is important for accountability (discussed earlier), keeping it in full makes for flexibility of reasoning, with multiple interpretations available for different applications and contexts. This facilitates achieving the *inconsistency principle*.

6.3. Interfaces

Our last set of principles relates to the design of user interfaces for scrutability. These are listed in Table V.

Interfaces for scrutable user models should enable the user to determine what information the model holds and the processes that formed it. If a model is large or complex, the interface needs to provide an *Overview* interface as a starting point and additional *Scrutiny* interfaces to enable the user to drill down on the details. This follows the recommendation to present visualizations of large collections of information using an overview first, with zoom and filter, then details on demand [Schneiderman 1996]. For example, we created the VLUM (Very Large User Model) interface [Apted et al. 2003; Uther and Kay 2003; Uther 2002; Kay and Lum 2005] for use in contexts where hundreds of aspects about them are modeled.

Taking the view that model values are determined from both external and internal evidence, the scrutiny interface should enable the user to see *inflow* to the model. An example of our work related to this principle was a concept mapping tool; this was a user model elicitation interface, based on concept mapping, an established technique for people to externalize their conceptual understanding [Novak 1990]. Our Verified

Concept Mapper (VCM) [Cimolino et al. 2004] enabled the user to lay out concepts to show hierarchical relationships (with more general ones higher) and link them to form propositions, such as *"a whale" is a "mammal"* or *"a whale" is a "fish"*. When the user was satisfied with his map, he clicked the analyze button and this presented him with:

—a list of the inferences the system had made and would store in the user model;
—things to check, where this was a list of propositions in the map or pointers to omissions.

So, for example, if the user had created the proposition *"a whale" is a "fish"*, this would be on the list of inferences. The creator of the system could also add a message to the list of things to check, such as *Are you sure you have linked "whale" to the right concepts?* This helped the user avoid modeling inaccurately, when he accidentally clicked the wrong concept and created an unintended proposition. The spirit of the design was to both ensure the user could see what was inferred about him and to help him verify that he had thought about key aspects to be used for building the model. In this sense, it was a form of verified input to the user model.

At the point where the user can see both the current value of a component of the model and the evidence for it, she should be able to *give a value*. This adds a piece of evidence for that component. Since the A-R representation keeps all such evidence, resolvers can treat this as a self-assessment, rather than ground truth; such evidence can be interpreted in combination with other evidence, for example, based on user behavior. The *ontology* principle calls for an interface that explains the meaning of the concepts modeled. For example, in our earlier scenario, it should explain what the *undo* command does (and hence what the model of knowledge of it means).

Interfaces for use of model have similar requirements to those just discussed for the viewing of the model itself. However, they must also take account of the context of use. This is likely to simplify the interface if the model may be used by multiple applications because the interface can focus on parts used by the current application. It may usefully restrict the displayed information to the current context. In other senses, the *overview* and *scrutiny of use* are as in the previous discussion. The interface support for *outflow* should enable the user to determine which parts of the model are released in general. That for the *outflow process* must do this for just those parts used in this context. It must also show how these are interpreted. A scrutiny interface for *use* should show what an application actually does in light of the user model values released to it. This constitutes a very pragmatic form of additional information about the truest meaning of the components of the model in terms of their actual use and the practical implications of their values for the user.

7. PERSONIS REALIZATION OF THE ARCHITECTURE: INTEGRATING SYSTEMS, REPRESENTATION, INFERENCE AND INTERFACES

This section illustrates the ways that we have created systems based upon the abstract principles and architecture of the previous two sections.[7] We summarize a selection of them in Table VI and refer to these as the "Personis" family (although individual versions and tools have various names.) Each row of the table refers to one system, with its key references. The italicized rows are for systems that were discussed in the case studies. We bring them together, ordered by the date of the main publication, to show the relationships between them. The dates are important because they reflect the influences of that time and our growing understanding of the challenges of supporting

[7]We have described the systems and representation evolution from a learning perspective elsewhere [Kay and Kummerfeld 2010].

Table VI. Personis Family Evolution

System name	Systems	Representation inference	Interface
um, sam editor study [Kay 1995]	Single Machine	Accretion-Resolution	Explanation Subsystem, Knowledge Mirror
sam editor coach [Cook et al. 1995]			Individual Usability
SASY 1 [Czarkowski and Kay 2000]		*Scrutable Hypertext*	*Subtle Link to Explanations*
Personis [Kay et al. 2002]	Server	Partial user models, personas	
VlUM/SIV [Uther 2002; Apted et al. 2003]			Large user model overview interface
VCM [Cimolino et al. 2004]			Scrutable Elicitation
Mobile [Brar 2004; Hitchens et al. 2005]	Signed Partial Models	Model Ontology definition	Mobile phone control interface
Personis [Carmichael et al. 2005]	Ubicomp Generalisation		
JITT [Holden et al. 2005]		*Scrutable Teaching Strategies*	*Scrutiny and control interfaces*
SASY II [Czarkowski 2006]			*Application-Level Scrutiny and Control*
PersonisAD [Assad et al. 2007; Carmichael 2009]	Distributed, Active	Re-active	Locator Interfaces
Locator [Niu and Kay 2008]		*Ontological Resolvers*	
Narcissus [Upton and Kay 2009]			User Model for Navigation
PersonisJ [Gerber et al. 2010]	Mobile		
PERSONAF [Niu and Kay 2010]		*Personal Ontologies*	*Scrutiny interfaces*

The left column is the system name and key publications. The second column summarizes the systems-level elements, the third has representation and inference mechanisms and the right column has interface aspects. Italicized systems were described in the earlier case studies.

scrutable user modeling. We begin with the systems aspects (the second column) then discuss the other two columns together as they are so tightly linked.

Systems

This dimension relates to the underlying operating systems, networks, and hardware, the latter two having seen important changes affecting the ways that one needs to create personalized systems. One of these has been due to the increase in the availability of mobile computing devices. This means that people today may well own a personal smart phone, tablet device, and portable computer and they may use multiple fixed desktop computers. We are seeing further transformations, with even richer personal digital ecosystems of devices. This is beginning to include various sensors, some worn or carried and others embedded in the environment. It may also include embedded devices such as tabletops and wall displays. There are important systems challenges if these are to provide effective personalization [Krüger et al. 2007] as well as new possibilities for capturing information about the user. The growing diversity and complexity of these ecosystems creates new challenges for scrutability and user control [Shilton 2009]. For example, they suggest the user may want to ask questions like:

—Where is my user model stored?
—How can I ensure that some user models (or parts of them) are stored only within my home?

Table VI begins with the *um toolkit* [Kay 1990, 1995]. Scrutability was a key driver for its design, leading to the basic accretion-resolution representation. It was intended to support learning by modeling use of the sam text editor. Learners used a single "large" (at the time) computer as her main (often only) computer. They interacted via a terminal and had a strict quota on filespace.

In this framework, the user model was stored as text files in a directory hierarchy in the user's own file space. This formed a directed acyclic graph of namespaces, we called *contexts*. For example, we created models of the student's knowledge of her main text editor (sam), her knowledge of unix and, for some users, their movie preferences. These were stored in a directory with each model in its own subdirectory. Within these were additional subdirectories, for subcontexts. At the leaves, files held the model's *components* and the evidence associated with each. Some aspects, such as knowledge about regular expressions, was common to the editor and unix and these aspects were linked.

This approach used the access controls of the underlying unix operating system, at the level of directories and files. Each person's model was restricted to the user. When the user started a personalized application, it ran under the user's own id and so could access the model. We instrumented the sam editor to log each user's activity. We took care in designing this to model only knowledge of the editor [Cook et al. 1995], avoiding any details of the content edited. The log was stored in the user's filespace. (Users were informed how to stop the modeling processes at this stage, by ensuring the automated removal of this log.) A process ran each night to analyze the log and add new evidence to the user model. Later, a coaching program used this model to identify aspects that the learner did not know and from these, chose coaching actions.

The term *context* for the nodes in the hierarchical model tree reflects that it holds parts of the model relevant to one context, such as the *sam editor* context. The same component name can have different meanings in different contexts. For example, we used this to deal with the somewhat different ways that regular expressions work in unix and the sam text editor. Each context has a set of associated *resolvers*. Each defines one way to interpret the evidence associated with a component. In summary, contexts partition and structure the user model components and define the resolvers allowed for determining the value of components.

In this implementation, each context file was stored as text that was intended to be human-readable. (Similar claims are made today when information is stored as xml but our format was simpler and clearer. Even so, it is notable that our approach shares the use of a text file format for storing information.) This was in the spirit of scrutability. While we created interfaces for scrutinizing user models our design ensured a user could choose to view the model directly. (This approach clearly allows the user to edit the model, potentially corrupting it.) The form of the text files matched closely to the formating of the scrutiny interfaces.

The Personis *server* [Kay et al. 2002] moved to a quite different architecture. A single server held the models for many users. A protocol enabled an application to establish a connection to the server, then to "tell" evidence to the server so that it could be stored with the relevant component and to "ask" for the value of a component. At this stage, we added the inflow and outflow control layers of Figure 18. This was an addition to the existing control in interpretation, based on the resolvers available for the model context. We also introduced two new selection mechanisms. The *partial model* defines a subset of the components within a single context. The other, more general mechanism

is a *persona*. It operates across contexts. In the earlier *um toolkit*, the context level defined the allowed resolvers and access; the persona allows for greater flexibility. It links a set of components with a resolver, to define one restricted view of the user. This means that different applications can access their own subset of the model and their own interpretation of the evidence associated with that.

To enable users to carry their models on a mobile phone, we created the Secure Persona Exchange (SPE) architecture. It enabled the user to load a model that was provided by a trusted authority [Hitchens et al. 2005]. For example, a university could provide a signed model for a student's enrollment or transcript. The user could then send this to a pervasive application offering a service, such as bookshop on the campus. The authority could also sign "partial models" so that the student could release just this subpart of the model. The user could alter her model if she wished. But then, that version would no longer be signed by the authority and so the application may not trust it. In this work, the trusted authority was also responsible for defining the user model ontology and sharing this with service providers to create their own applications.

Pervasive computing has created a substantial body of research that aims to provide new services for every day life by making use of many sensors and other devices in a user's environment. The sensors often collect information that aims to model people. For example, it may capture data about their physical activity. There have been many, many architectures defined, and some implemented, for managing pervasive computing's sensor data and applications. It is somewhat surprising that these had not been linked to work on user modeling frameworks until we adapted the accretion-resolution representation to model devices and places as well as people [Carmichael et al. 2005]. This meant that the Personis framework could represent and reason about all of these (people, places, and devices) in a consistent manner.

The next stage of our work enabled Personis to support other key functions of a pervasive computing framework [Assad et al. 2007]. It could be distributed, with one model stored on one machine, another model on a different machine, and so on. Of course, the system had to be able to find all relevant models as needed. This version of Personis was also active in the sense that the model could drive actions in the outside world. So, for example, it could start a music system playing and control it.

Our mobile phone version of Personis [Gerber et al. 2010] is, conceptually, a cut-down version of the main Personis framework. (It is a reimplementation in Java.) In terms of the core user modeling, this is partly an interim technology as mobile phones will soon have the power and flexibility to run the full Personis. In addition, it supports the dynamic download of arbitrary personalized applications. It only allows an application to access the model via a trusted "stub". (This could be managed following an app-store model.) Then the mobile user can download untrusted applications that operate in conjunction with the appropriate stub. The framework's security mechanism runs the untrusted app in a sandbox, so that it cannot access either the user model or the network. The trusted stub acts as a mediator between untrusted application and user model.

User model representation, inference and interfaces. This section outlines the key ideas in the tightly linked technical and interface issues. We draw heavily on the systems from our earlier case studies. (These are italicized in Table VI.) Taking the *simplicity* principle from Table IV, we began by designing and building systems to be as simple as possible, but still useful for personalization. We then evaluated these, both from a technical viewpoint and to assess effectiveness of the interfaces for scrutability. This meant testing whether people could actually scrutinize the system to answer questions like those in Section 2. We typically needed several iterative cycles of refinement, both at technical and interface level. We now discuss the ways that we have

tackled the representation and user interface challenges of scrutable personalization, summarized in Table VI. Although the table is in chronological order, this discussion is organized in themes and in the discussion of interface issues, we refer to the principles in Table V.

The goal of a simple, explainable user model representation led to an analysis of the forms of evidence [Kay 1995] and design of the accretion-resolution representation. In the context of the sam text editor coaching system, we created long-term models of users and interfaces that enabled users to see these and add evidence about their self-perception of their knowledge: the give value principle in Table V. The interface had an explanation subsystem to provide personalized explanations of the concepts in the model (the *ontology* principle in Table V). This work also explored the notion of individual usability, discussed in Section 4 as a role for user models.

The next major step was the SASY I [Czarkowski and Kay 2000], that enabled the user to scrutinize an adaptive hypertext, as well as the model driving its personalization (Table V – *use* principle). This had subtle links to explanations, and people did not notice them. As we have described, SASY II [Czarkowski 2006] made the personalization much more obvious. Usability studies as well as the authentic field trial indicated that it was quite successful in enabling people to determine what had been personalization, what had been omitted from the personalized view, how the user model affected this and how to alter the model. It did, however, have only a simple form of personalization with parts of the page selectively presented, under the control of user-model-based rules.

We now consider the main stages in the Personis user modeling framework. Like the um toolkit, this uses the accretion-resolution representation. The Personis server [Kay et al. 2002] introduced representations of partial user models and personas. These were subsets of the model for use by particular applications. As indicated in Table VI, there was no user interface for these. This was partly addressed in the first exploration of mobile user models on phone [Brar 2004; Hitchens et al. 2005]. This work tackled one of the ongoing challenges of interoperability for user modeling, the definition of ontologies that can be understood by multiple applications [Carmagnola et al. 2011]. It also provided a phone interface for defining personas and small-scale usability studies indicated people could control this to allow selective sharing of their user model [Brar 2004] (*outflow* principle in Table V). The Ubicomp generalization taking the same accretion-resolution representation to modeling people, places, and devices, was designed for scrutability and was used for the first versions of the Locator system described in the last case study. It had a limited scrutiny interface, with a link to the recent evidence used to infer a person's location. This was addressed with PersonisAD when a carefully crafted control interface was created [Carmichael 2009]. User studies indicated that people were able to organize their friends into groups, and then control the choice of evidence filter and resolver for each group (*outflow* and *outflow processes* in Table V). The last Personis version in Table VI incorporated sophisticated ontological reasoning in the resolvers. As we showed in the Locator case study, this provided a scrutiny interface that enabled people to see why people were presented (or not) and to see explanations of the personalization of location names.

We now turn to VlUM [Uther 2002; Uther and Kay 2003], a viewer for very large user models. This was designed to enable people to gain an overview of a large user model, with hundreds of elements modeled (the *overview* principle in Table V). Its successor, SIV [Lum 2007] used a lightweight ontology that was automatically generated by analyzing online dictionaries [Apted et al. 2003]. As already discussed, this approach was chosen to exploit the dictionary definitions to explain concepts in the user model (the *ontology* principle in Table V). It was used in a programming teaching environment [Kay et al. 2007] and in an HCI course [Lum 2007].

We have already introduced the remaining parts of Table VI. VCM [Cimolino et al. 2004] was discussed as an example of the *inflow* principle in Table V. It elicited information from the user, as many applications do. However, unlike most such applications, it showed the user what it intended to infer about him. We discussed JITT in the second case study. It supported scrutiny of the user model and control over the resolver and teaching strategy, supporting the *outflow processes* principle in Table V. We used the Narcissus system in Section 4 to illustrate the role of a user model for navigation. It does this by providing a form of overview of the user model (Table V, *overview* principle).

Overall, it is clear that Table VI has several remaining blanks in the interface column indicating much still to be done. At the same time, the systems described illustrate forms of each of the interface principles for scrutable user modeling and personalization summarized in Table V.

8. REFLECTIONS AND REVISIONS

A core goal of this article has been to share our vision for scrutability in personalization and the benefits it has the potential to offer. We acknowledge that these benefits need to justify the effort and cost of creating personalized systems in a manner that supports scrutability. However, that cost can be reduced if we can build our understanding of the task, and establish a set of tools for creating scrutably personalized systems. That has been a key goal of much of our work summarized in this article.

We began this article by identifying key problems of personalization that scrutable user modeling can help address. These related to: privacy, invisibility, errors in user models, wasted user models, and lack of user control. To explain our vision, we presented two scenarios, one in a learning context and the other for personalized news. We identified several key classes of questions that people may ask about personalized systems and their user models. We then presented three detailed case studies of our work. The first, SASY, enabled people to determine how a Web page had been adapted to them and to scrutinize and alter the user model. The second, JITT, enabled people to explore their user model, down to the detailed evidence and to control the choice of resolver that interprets the evidence as well as the teaching strategy controlling the personalization. Locator, the third case study, made use of pervasive computing sensors and a rich ontological reasoning mechanism to personalize a location information display. It made use of the unified representation to model the sensors, places, and users. These case studies were presented to provide concrete examples of elements of our vision. Section 4 then moved to the broader roles that scrutable user models can play, as further motivation for the potential benefits of tackling the challenges involved in supporting scrutability.

The next two sections drew upon our own work and a broad range of other user modeling research to present design principles and a high-level, evidence-based architecture. Researchers and systems builders can apply these, or at least consider them, when they design user models. We summarized a selection of our own systems, in terms of the research towards creating systems, representation, and inference mechanisms and the critical interfaces for scrutable user models. We now can identify key challenges yet to be met.

Personal digital ecosystems, "client-side" user model clouds. We are seeing a fast rise in the numbers of devices in people's personal digital ecosystems. For example, a recent report [CISCO 2011] points to a dramatic rise in "tablets, mobile phones, connected appliances and other smart machines" making for a doubling of networked devices from 2010 to 2015, with many of these wirelessly connected. This will enable new forms of personalization. It will also mean that parts of the user model will need to be stored on various devices as well as on servers that are accessible from a person's

devices. Conceptually, these user models will reside in a "personal user model cloud". If people are to become masters of their own user models, these will need to become first-class citizens, with scrutiny and control interfaces. This matches our extended view of "client-side" personalization to protect privacy based on the architecture described in Section 7. It will be challenging to create effective interfaces for people to control such user models.

Sense making, sharing and scrutability. When the user needs to make sense of his own user model, he may need comparable information for other people. For example, in our models of learners [Uther and Kay 2003; Kay and Lum 2005] one valuable form of the model showed the learner's progress compared against the class as a whole, or against the top group in the class. In more general contexts, such as personal informatics [Li et al. 2010], this may mean that aggregated stores are needed. A reciprocity principle often applies in such cases, where the individual is allowed to see comparative information if he is willing to release his own data for aggregation. This creates the need for effective scrutiny and control of the parts of the user model to be shared. It also may call for mechanisms to blur the information released [Berkovsky et al. 2007].

Scrutability toolkits for application builders, making for familiarity and pervasiveness of scrutiny mechanisms. One of the challenges we faced in creating scrutability interfaces is that users did not expect to be able to scrutinize the personalization [Czarkowski and Kay 2006; Czarkowski 2006]. In fact, people were so used to errors in personalization that they were unsurprised by them. They simply accepted them. It may be too much to hope for such expectations to change in the near future. Beyond that, it will be far easier for people to scrutinize their user models and personalization if scrutable user modeling becomes more common and if consistency in scrutiny interface elements emerges. Toolkits for scrutable personalization will help here and also ease the burden on programmers to support scrutability.

The increasingly pressing need for scrutability. Personalization is likely to play an increasing role in people's lives. There is already widespread personalization on commercial Web sites and this is likely to spread both in that context and in emerging forms on various devices. We are beginning to see explanations in widespread Web interfaces when the user is asked for personal information. Typically this is a link next to a particular field, so that the user can see why this information is needed. That information often includes a link to the privacy policy for the Web site. Our vision is for a deeper form of support for scrutability to become the norm. This is important on many levels, from the most basic requirement of ensuring that a person has control over her personal information and for privacy as well as the many potential benefits that a scrutable user model can offer for learning and navigating our increasingly complex information spaces.

ACKNOWLEDGMENTS

We thank Andrew Claypham for reviewing the final draft.

REFERENCES

Ahn, J.-w., Brusilovsky, P., Grady, J., He, D., and Syn, S. Y. 2007. Open user profiles for adaptive news systems: Help or harm? In *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*. ACM, New York, 11–20.

Apted, T. and Kay, J. 2004. Mecureo ontology and modelling tools. *Int. J. Contin. Engin. Educ. Lifelong Learn. 14,* 3, 191–211.

Apted, T., Kay, J., Lum, A., and Uther, J. 2003. Visualisation of ontological inferences for user control of personal web agents. In *Proceedings of 7th International Conference on Information Visualization (IV*

'03), E. Banissi, K. Borner, C. Chen, G. Clapworthy, C. Maple, A. Lobben, C. Moore, J. Roberts, A. Ursyn, and J. Zhang, Eds. IEEE Computer Society, 306–311.

Assad, M., Carmichael, D., Kay, J., and Kummerfeld, B. 2007. Personisad: Distributed, active, scrutable model framework for context-aware services. In *Pervasive Computing*, A. LaMarca, M. Langheinrich, and K. Truong, Eds. Lecture Notes in Computer Science, vol. 4480. Springer, 55–72.

Barkhuus, L. and Dey, A. 2003. Is context-aware computing taking control away from the user? three levels of interactivity examined. In *Proceedings of the Conference on Ubiquitous Computing (UbiComp '03)*, A. Dey, A. Schmidt, and J. McCarthy, Eds. Lecture Notes in Computer Science, vol. 2864. Springer, 149–156.

Berkovsky, S., Eytani, Y., Kuflik, T., and Ricci, F. 2007. Enhancing privacy and preserving accuracy of a distributed collaborative filtering. In *Proceedings of the ACM Conference on Recommender Systems*. ACM, 9–16.

Bleiholder, J. and Naumann, F. 2009. Data fusion. *ACM Comput. Surv. 41*, 1:1–1:41.

Brar, A. 2004. Privacy and security in ubiquitous personalised applications. Honours thesis, Unversity of Sydney.

Browne, D., Totterdell, P., and Norman, M. 1990. *Adaptive User Interfaces*. Academic Press.

Brusilovsky, P. 1996. Methods and techniques of adaptive hypermedia. *User Model. User-Adapt. Interact. 6,* 2, 87–129.

Brusilovsky, P., Kobsa, A., and Nejdl, W. 2007. *The Adaptive Web: Methods and Strategies of Web Personalization*. Springer.

Brusilovsky, P. and Millán, E. 2007. User models for adaptive hypermedia and adaptive educational systems. In *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Lecture Notes in Computer Science, vol. 4321. Springer, 3–53.

Bull, S., Brna, P., and Pain, H. 1995. Extending the scope of the student model. *User Model. User-Adapt. Interact. 5,* 1, 45–65.

Bull, S. and Gardner, P. 2009. Highlighting learning across a degree with an independent open learner model. In *Artificial Intelligence in Education*. IOS Press, 275–282.

Bull, S. and Kay, J. 2007. Student models that invite the learner in: The SMILI open learner modelling framework. *Int. J. Artif. Intell. 17,* 2, 89–120.

Bull, S. and Kay, J. 2008. Metacognition and open learner models. In *Proceedings of the 3rd Workshop on Meta-Cognition and Self-Regulated Learning in Educational Technologies, at ITS'08, Intelligent Tutoring Systems*. 7–20.

Bull, S. and Mabbott, A. 2006. 20000 inspections of a domain-independent open learner model with individual and comparison views. In *Proceedings of the Conference on Intelligent Tutoring Systems (ITS'06)* M. Ikeda, K. D. Ashley, and T.-W. Chan, Eds. Vol. 4053, 422–432.

Bull, S., Mabbott, A., and Abu Issa, A. 2007. Umpteen: Named and anonymous learner model access for instructors and peers. *Int. J. Artif. Intell. Educ. 17,* 3, 227–253.

Carmagnola, F., Cena, F., and Gena, C. 2011. User model interoperability: a survey. *User Model. User-Adapt. Interact. 21*, 285–331.

Carmagnola, F. and Dimitrova, V. 2008. An evidence-based approach to handle semantic heterogeneity in interoperable distributed user models. In *Adaptive Hypermedia and Adaptive Web-Based Systems*. Springer, 73–82.

Carmichael, D. 2009. Myplace: Supporting scrutability and user control in location modelling. Ph.D. thesis, University of Sydney.

Carmichael, D. J., Kay, J., and Kummerfeld, R. J. 2005. Consistent modelling of users, devices and sensors in a ubiquitous computing environment. *User Model. User-Adapt. Interact. 15,* 3-4, 197–234.

Chellappa, R. and Sin, R. 2005. Personalization versus privacy: An empirical examination of the online consumers dilemma. *Inf. Technol. Manag. 6,* 2, 181–202.

Cimolino, L., Kay, J., and Miller, A. 2004. Concept mapping for eliciting verified personal ontologies. *Int. J. Contin. Engin. Educ. Life-Long Learn. 14,* 3, 212–228.

CISCO. 2011. Cisco visual networking index: Forecast and methodology, White Paper, 2010–2015.

Clancey, W. 1983. The epistemology of a rule-based expert system–A framework for explanation. *Artif. Intell. 20,* 3, 215–251.

Cook, R. and Kay, J. 1994. The justified user model: A viewable, explained user model. In *Proceedings of the 4th International Conference on User Modeling (UM'94)*, A. Kobsa and D. Litman, Eds. Mitre, 145–150.

Cook, R., Kay, J., Ryan, G., and Thomas, R. 1995. A toolkit for appraising the long-term usability of a text editor. *Softw. Qual. J. 4,* 2, 131–154.

Core, M., Lane, H., Van Lent, M., Gomboc, D., Solomon, S., and Rosenberg, M. 2006. Building explainable artificial intelligence systems. In *Proceedings of the National Conference on Artificial Intelligence*. Vol. 21.

Cramer, H., Evers, V., Ramlal, S., van Someren, M., Rutledge, L., Stash, N., Aroyo, L., and Wielinga, B. 2008. The effects of transparency on trust in and acceptance of a content-based art recommender. *User Model. User-Adapt. Interact. 18,* 5, 455–496.

Csinger, A. 1995. User models for intent-based authoring. Ph.D. thesis, University of British Columbia.

Czarkowski, M. 2006. A scrutable adaptive hypertext. Ph.D. thesis, University of Sydney.

Czarkowski, M. and Kay, J. 2000. Bringing scrutability to adaptive hypertext teaching. In *Proceedings of the Conference on Intelligent Tutoring Systems (ITS'00)*, G. Gauthier, C. Frasson, and K. VanLehn, Eds. 423–432.

Czarkowski, M. and Kay, J. 2006. Giving learners a real sense of control over adaptivity, even if they are not quite ready for it yet. In *Advances in Web-Based Education: Personalized Learning Environments.* IDEA Information Science Publishing, Hershey, PA, USA, 93–125.

Dimitrova, V. 2003. STyLE-OLM: Interactive open learner modelling. *Int. J. Artif. Intell. Educ. 13,* 1, 35–78.

Gerber, S., Fry, M., Kay, J., Kummerfeld, B., Pink, G., and Wasinger, R. 2010. PersonisJ: mobile, client-side user modelling. In *Proceedings of the Conference on User Modeling, Adaptation and Personalisation (UMAP'10)*. Springer, 111–122.

Glass, A., McGuinness, D., and Wolverton, M. 2008. Toward establishing trust in adaptive agents. In *Proceedings of the International Conference on Intelligent User Interfaces*. Vol. 8. 227–236.

Herlocker, J. L., Konstan, J. A., and Riedl, J. 2000. Explaining collaborative filtering recommendations. In *Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW '00)*, ACM, New York, 241–250.

Hilbert, D. and Redmiles, D. 2000. Extracting usability information from user interface events. *ACM Comput. Surv. 32,* 4, 384–421.

Hitchens, M., Kay, J., Kummerfeld, R. J., and Brar, A. 2005. Secure identity management for pseudo-anonymous service access. In *Proceedings of the 2nd International Conference on Security in Pervasive Computing (SPC 2005)*, D. Hutter, Ed. Lecture Notes in Computer Science, vol. 3450. Springer, 48–55.

Holden, S., Kay, J., Poon, J., and Yacef, K. 2005. Workflow-Based personalised document delivery (just-in-time training system). *J. Interact. Learn. 4,* 1, 131–148.

Hook, K. 2000. Steps to take before intelligent user interfaces become real. *Interact. Comput. 12,* 4, 409–426.

Hook, K., Karlgren, J., Waern, A., Dahlback, N., Jansson, C., Karlgren, K., and Lemaire, B. 1996. A glass box approach to adaptive hypermedia. *User Model. User-Adapt. Interact. 6,* 2, 157–184.

Horvitz, E., Breese, J., Heckerman, D., Hovel, D., and Rommelse, K. 1998. The Lumiere project: Bayesian user modeling for inferring the goals and needs of software users. In *Proceedings of the 14th Conference on Uncertainty in Artificial Intelligence (UAI'98)*. Morgan Kaufmann Publishers, San Francisco, CA, 256–265.

Iachello, G. and Hong, J. 2007. End-User privacy in human-computer interaction. *Found. Trends Hum.-Comput. Interact. 1,* 1, 1–137.

Jameson, A. and Schwarzkopf, E. 2006. Pros and cons of controllability: An empirical study. In *Adaptive Hypermedia and Adaptive Web-Based Systems*. Springer, 193–202.

Kay, J. 1990. UM: A user modelling toolkit. In *the 2nd International User Modeling Workshop*.

Kay, J. 1995. The um toolkit for cooperative user modelling. *User Model. User-Adapt. Interact. 4*, 149–196.

Kay, J. 1997. Learner know thyself: Student models to give learner control and responsibility. In *Proceedings of International Conference on Computers in Education*. 17–24.

Kay, J. 1999. A scrutable user modelling shell for user-adapted interaction. Ph.D. thesis, University of Sydney.

Kay, J. 2000a. Accretion representation for scrutable student modelling. In *Proceedings of the Conference on Intelligent Tutoring Systems (ITS'00)*, G. Gauthier, C. Frasson, and K. VanLehn, Eds. 514–523.

Kay, J. 2000b. Invited keynote: Stereotypes, student models and scrutability. In *Proceedings of the Conference on Intelligent Tutoring Systems (ITS'00)*, G. Gauthier, C. Frasson, and K. VanLehn, Eds. Springer, 19–30.

Kay, J. and Kummerfeld, B. 2010. Tackling HCI challenges of creating personalised, pervasive learning ecosystems. In *Sustaining TEL: From Innovation to Learning and Practice 5th European Conference on Technology Enhanced Learning, (EC-TEL '10)*, M. Wolpers, P. A. Kirschner, M. Scheffel, S. Lindstaedt, and V. Dimitrova, Eds. Springer, 1–16.

Kay, J., Kummerfeld, B., and Lauder, P. 2002. In *Proceedings of the 2nd International Conference on Adaptive Hypermedia and Adaptive Web-Based Systems (AH '02)*, P. D. Bra, P. Brusilovsky, and R. Conejo, Eds. Vol. 2347. Springer, 203–212.

KAY, J. AND KUMMERFELD, R. 1994. An individualised course for the C programming language. In *Proceedings of 2nd International World Wide Web Conference*. 17–20.

KAY, J., LI, L., AND FEKETE, A. 2007. Learner reflection in student self-assessment. In *Proceedings of the 9th Australasian Computing Education Conference (ACE '07)*. Australian Computer Society, Sydney, 89–95.

KAY, J. AND LUM, A. 2005. Exploiting readily available web data for reflective student models. In *Proceedings of the Conference Artificial Intelligence in Education (AIED '05)*. IOS Press, Amsterdam, The Netherlands, 338–345.

KAY, J. AND THOMAS, R. C. 1995. Studying long-term system use. *Comm. ACM 38,* 7, 61–69.

KOBSA, A. 2001. Generic user modeling systems. *User Model. User-Adapt. Interact. 11,* 1, 49–63.

KOBSA, A. 2002. Personalized hypermedia and international privacy. *Comm. ACM 45,* 5, 64–67.

KOBSA, A. 2007. Privacy-Enhanced web personalization. In *The Adaptive Web: Methods and Strategies of Web Personalization*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Springer, 628–670.

KOBSA, A. AND FINK, J. 2006. An LDAP-based user modeling server and its evaluation. *User Model. User-Adapt. Interact. 16,* 2, 129–169.

KOBSA, A., KOENEMANN, J., AND POHL, W. 2001. Personalised hypermedia presentation techniques for improving online customer relationships. *The Knowl. Engin. Rev. 16,* 02, 111–155.

KOBSA, A. AND WAHLSTER, W. 1989. *User Models in Dialog Systems*. Springer.

KOIDL, K. AND CONLAN, O. 2008. Engineering information systems towards facilitating scrutable and configurable adaptation. *Adapt. Hypermedia and Adapt. Web-Based Syst. 5149*, 405–409.

KONO, Y., IKEDA, M., AND MIZOGUCHI, R. 1992. To contradict is human-student modeling of inconsistency. In *Proceedings of the Conference on Intelligent Tutoring Systems (ITS'92)*. Springer, 451–458.

KRÜGER, A., BAUS, J., HECKMANN, D., KRUPPA, M., AND WASINGER, R. 2007. Adaptive Mobile Guides. In *The Adaptive Web: Methods and Strategies of Web Personalization*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Springer, 521–549.

LANGHEINRICH, M. 2002. A privacy awareness system for ubiquitous computing environments. In *4th International Conference on Ubiquitous Computing (Ubicomp '02)*, G. Borriello and L. E. Holmquist, Eds. Lecture Notes in Computer Science, vol. 2498. Springer, 237–245.

LEE, S. AND BULL, S. 2008. An open learner model to help parents help their children. *Technol. Instruct. Cogn. and Learn. 6,* 1, 29.

LI, I., DEY, A., AND FORLIZZI, J. 2010. A stage-based model of personal informatics systems. In *Proceedings of the 28th International Conference on Human Factors in Computing Systems (CHI '10)*. 557–566.

LIM, B., DEY, A., AND AVRAHAMI, D. 2009. Why and why not explanations improve the intelligibility of context-aware intelligent systems. In *Proceedings of the 27th International Conference on Human Factors In Computing Systems (CHI'09)*. ACM New York, 2119–2128.

LIM, B. Y. AND DEY, A. K. 2009. Assessing demand for intelligibility in context-aware applications. In *Proceedings of the 11th International Conference on Ubiquitous Computing (Ubicomp '09)*. ACM, New York, 195–204.

LIM, B. Y. AND DEY, A. K. 2010. Toolkit to support intelligibility in context-aware applications. In *Proceedings of the 12th ACM International Conference on Ubiquitous Computing (Ubicomp '10)*. ACM, New York, 13–22.

LINTON, F. AND SCHAEFER, H. 2000. Recommender systems for learning: Building user and expert models through long-term observation of application use. *User Model. User-Adapt. Interact. 10,* 2, 181–208.

LUM, A. 2007. Light-Weight ontologies for scrutable user modelling. Ph.D. thesis, University of Sydney.

MITROVIC, A. AND MARTIN, B. 2006. Evaluating the effects of open student models on learning. In *Adaptive Hypermedia and Adaptive Web-Based Systems*. Springer, 296–305.

NIU, W. AND KAY, J. 2008. Location conflict resolution with an ontology. In *Proceedings of the 6th International Conference on Pervasive Computing (Pervasive '08)*. Springer, 162–179.

NIU, W. T. AND KAY, J. 2010. PERSONAF: Framework for personalised ontological reasoning in pervasive computing. *User model. User-Adapt. Interact.: The J. Personal. Res. 20,* 1, 1–40.

NOVAK, J. 1990. Concept maps and vee diagrams: Two metacognitive tools to facilitate meaningful learning. *Instruct. Sci. 19,* 1, 29–52.

ORWANT, J. 1994. Heterogeneous learning in the doppelgänger user modeling system. *User Model. User-Adapt. Interact. 4,* 2, 107–130.

PALEN, L. AND DOURISH, P. 2003. Unpacking "privacy" for a networked world. In *Proceedings of the Conference on Human Factors in Computing Systems (CHI'03)*, 129–136.

PARAMYTHIS, A., WEIBELZAHL, S., AND MASTHOFF, J. 2010. Layered evaluation of interactive adaptive systems: Framework and formative methods. *User Model. User-Adapt. Interac.* 1–71.

PARISER, E. 2011. *The Filter Bubble: What the Internet is Hiding from You*. Penguin.

PRICE, B., ADAM, K., AND NUSEIBEH, B. 2005. Keeping ubiquitous computing to yourself: A practical model for user control of privacy. *Int. J. Human-Comput. Stud. 63,* 1-2, 228–253.

RICH, E. 1979. User modeling via stereotypes. *Cogn. Sci. 3*, 329–354.

RICH, E. 1983. Users are individuals: Individualizing user models. *Int. J. Man-Mach. Stud. 18*, 199–214.

SCHNEIDERMAN, B. 1996. The eyes have it: A task by data type taxonomy for information visualizations. In *Proceedings of the IEEE Symposium on Visual Languages.* IEEE Computer Society, 336–343.

SELF, J. 1999. Open Sesame?: Fifteen variations on the theme of openness in learning environments. *Int. J. Artif. Intell. Educ. 10*, 1020–1029.

SELLEN, A. J. AND WHITTAKER, S. 2010. Beyond total capture: A constructive critique of lifelogging. *Comm. ACM 53,* 5, 70–77.

SHILTON, K. 2009. Four billion little brothers?: Privacy, mobile phones, and ubiquitous data collection. *Queue 7,* 7, 40–47.

SPIEKERMANN, S. 2008. *User Control in Ubiquitous Computing: Design Alternatives and User Acceptance (Habilitation)*. Shaker.

SPIEKERMANN, S. AND CRANOR, L. 2009. Engineering privacy. *IEEE Trans. Softw. Engin. 35,* 1, 67–82.

TAMIR, P. 1984. What do learning theories and research have to say to practitioners in science education? In *Research and Development in Higher Education*, J. Lublin, Ed. Vol. 7. 162–166.

TINTAREV, N. AND MASTHOFF, J. 2007. Effective explanations of recommendations: User-Centered design. In *Proceedings of the ACM Conference on Recommender Systems*. ACM, 153–156.

TOCH, E., WANG, Y., AND CRANOR, L. F. 2012. Personalization and privacy: A survey of privacy risks and remedies in personalization-based systems. *User Model. User-Adapt. Interact. 20,* 1, 203–220.

UPTON, K. AND KAY, J. 2009. Narcissus: Interactive activity mirror for small groups. In *Proceedings of the Conference on User Modeling, Adaptation and Personalisation (UMAP'09)*, 54–65.

UTHER, J. 2002. On the visualisation of large user models in web based systems. Ph.D. thesis, University of Sydney.

UTHER, J. AND KAY, J. 2003. Vlum, a web-based visualisation of large user models. In *Proceedings of the Conference on User Modeling (UM '03)*, P. Brusilovsky, A. Corbett, and F. de Rosis, Eds. Springer, 198–202.

WANG, Y. AND KOBSA, A. 2007. Respecting users individual privacy constraints in web personalization. In *Proceedings of the Conference on User Modeling Adaptation and Personalization (UMAP'07)*, 157–166.

WANG, Y. AND KOBSA, A. 2009. Performance evaluation of a privacy-enhancing framework for personalized websites. In *Proceedings of the Conference on User Modeling, Adaptation, and Personalization (UMAP'09)*, 78–89.

WEBER, G. AND BRUSILOVSKY, P. 2001. ELM-ART: An adaptive versatile system for web-based instruction. *Int. J. Artif. Intell. Educ. 12,* 4, 351–384.

WESTIN, A. 2003. Social and political dimensions of privacy. *Contemp. Perspect. Priv. Social, Psychol. Polit. 59,* 2, 431–453.

WOLF, G. May, 2, 2010. The data-driven life. *The New York Times 28*, MM38 of the Sunday Magazine.

YUDELSON, M., BRUSILOVSKY, P., AND ZADOROZHNY, V. 2007. A user modeling server for contemporary adaptive hypermedia: An evaluation of the push approach to evidence propagation. In *Proceedings of the Conference on User Modeling (UM '07)* Vol. 4511, 27–36.

ZAPATA-RIVERA, D., HANSEN, E., SHUTE, V., UNDERWOOD, J., AND BAUER, M. 2007. Evidence-based approach to interacting with open student models. *Int. J. Artif. Intell. Educ. 17,* 3, 273–303.

ZAPATA-RIVERA, J. AND GREER, J. 2004. Inspectable bayesian student modelling servers in multi-agent tutoring systems. *Int. J. Human-Comput. Stud. 61,* 4, 535–563.

ZASLOW, J. 2002. If TiVo thinks you are gay, here's how to set it straight. *Wall St. J. 26*.

Received May 2011; revised February 2012; accepted March 2012